

Now can you give me the proto of the rxu connectivity which publishes the tcp, joint values and also KPP values?

did we decided that both TCP/ joint values and KPP values will be pulished over two different topic and will have two different proto definitions?

Yes — that is the design we had converged on.

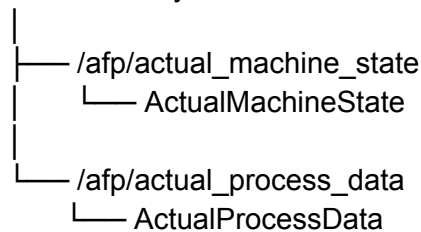
For **RXU Connectivity**, I would publish:

- **Topic 1:** actual TCP + joint state
- **Topic 2:** actual KPP/process values

with **two different protobuf messages**, but both reusing the same common manufacturing context/header.

That is cleaner because TCP/joints and KPPs can have different update rates and different consumers.

RXU Connectivity



A good split is:

```
// common.proto  
// connectivity.proto
```

common.proto

```
syntax = "proto3";
```

```
package rxu.afp.common;
```

```
import "google/protobuf/timestamp.proto";
```

```

//
=====
====
// Manufacturing context
//
// Identifies where this measurement belongs in the manufacturing process.
// Connectivity publishes this context together with the actual values when
// this information is available from the AFP system.
//
=====
====

message ManufacturingContext {

    string machine_id = 1;

    string product_id = 2;

    string part_id = 3;

    string run_id = 4;

    string program_id = 5;

    string ply_id = 6;

    string course_id = 7;

    optional string strip_id = 8;

    // Current executing NC block, when available.
    optional uint32 nc_block_id = 9;

    // Current position/progress along the active course/path.
    // SI unit: metres.
    optional double path_progress_m = 10;
}

//
=====
====
// Common header

```

```

//
=====

message SampleHeader {

    // Timestamp from the original controller/sensor/source whenever possible.
    google.protobuf.Timestamp source_timestamp = 1;

    // Timestamp when RXU Connectivity publishes/processes this sample.
    google.protobuf.Timestamp connectivity_timestamp = 2;

    uint64 sequence_id = 3;

    ManufacturingContext context = 4;
}

//
=====

// Geometry
//
=====

message Vector3 {
    double x = 1;
    double y = 2;
    double z = 3;
}

message Quaternion {
    double x = 1;
    double y = 2;
    double z = 3;
    double w = 4;
}

message Pose {

    // Position normally represented in metres.

```

```

Vector3 position = 1;

Quaternion orientation = 2;

// Example:
// "machine_base"
// "part_frame"
// "panel_frame"
string frame_id = 3;
}

//
=====
====
// Machine execution state
//
=====
====

enum ExecutionState {

    EXECUTION_STATE_UNSPECIFIED = 0;

    IDLE = 1;

    PROGRAM_LOADED = 2;

    RUNNING = 3;

    PAUSED = 4;

    COMPLETED = 5;

    ABORTED = 6;

    FAULT = 7;
}

```

Then `connectivity.proto` uses these common definitions.

connectivity.proto

```
syntax = "proto3";
```

```
package rxu.afp.connectivity;
```

```
import "common.proto";
```

```
//
```

```
=====
```

```
====
```

```
// JOINT STATE
```

```
//
```

```
=====
```

```
====
```

```
message JointState {
```

```
    // Example:
```

```
    // "joint_1"
```

```
    // "joint_2"
```

```
    // "gantry_x"
```

```
    string joint_name = 1;
```

```
    // Rotary joint: rad
```

```
    // Linear axis: m
```

```
    double position = 2;
```

```
    // Rotary joint: rad/s
```

```
    // Linear axis: m/s
```

```
    optional double velocity = 3;
```

```
    // Nm for rotary joint or N for linear axis if available.
```

```
    optional double effort = 4;
```

```
}
```

```
//
```

```
=====
```

```
====
```

```
// MESSAGE 1
```

```
// ACTUAL TCP + JOINT DATA
```

```
//
```

```

// Suggested topic:
// /afp/actual_machine_state
//
=====
=====

message ActualMachineState {

    // Timestamp + run/part/ply/course/program context.
    rxu.afp.common.SampleHeader header = 1;

    // Measured/actual TCP.
    rxu.afp.common.Pose actual_tcp = 2;

    // Robot / gantry joints.
    repeated JointState joints = 3;

    // Optional actual linear TCP velocity.
    // Unit: m/s.
    optional rxu.afp.common.Vector3 tcp_linear_velocity = 4;

    // Optional actual angular TCP velocity.
    // Unit: rad/s.
    optional rxu.afp.common.Vector3 tcp_angular_velocity = 5;

    // Actual machine/feed/laying speed if provided by controller.
    // Unit: m/s.
    optional double actual_process_speed_mps = 6;

    rxu.afp.common.ExecutionState execution_state = 7;
}

//
=====
=====
// TOW-SPECIFIC PROCESS DATA
//
=====
=====

message TowProcessData {

    // Example:

```

```

// TOW_01 ... TOW_48
string tow_id = 1;

// Optional physical acquisition channel.
// Example: "CH17"
optional string channel_id = 2;

// Unit: N
optional double tension_n = 3;

// Only include if tow-specific temperature is actually available.
// Unit: degree Celsius
optional double temperature_c = 4;

// Only include if tow-specific pressure is available.
// Unit: Pa
optional double pressure_pa = 5;

// Status signals where provided by the upstream AFP/inspection system.
optional bool tow_present = 6;

optional bool tow_broken = 7;

optional bool tow_laid = 8;
}

//
=====
====
// MESSAGE 2
// ACTUAL KPP / PROCESS DATA
//
// Suggested topic:
// /afp/actual_process_data
//
=====
====

message ActualProcessData {

// Same manufacturing context concept as machine-state message.
rxu.afp.common.SampleHeader header = 1;

```

```

// -----
// Head/process-level KPP values
// -----

// Unit: N
optional double compaction_force_n = 2;

// Unit: Pa
optional double compaction_pressure_pa = 3;

// Unit: degree Celsius
optional double process_temperature_c = 4;

// Unit: m/s
optional double laying_speed_mps = 5;

// -----
// Tow-specific measurements
//
// Example: 48 entries for a 48-tow AFP head.
// -----

repeated TowProcessData tow_data = 6;
}

```

Why two topics?

Because they are logically different streams.

For example:

ActualMachineState



TCP + joints



TCP/Joint Deviation RXU

while:

ActualProcessData



Tension / pressure / temperature / speed



KPP Trend RXU



Process Anomaly RXU

They may also run at different frequencies:

TCP/joints maybe 100 Hz

Tow tension maybe 30–50 Hz

Temperature maybe 5–10 Hz

So putting them into one large message would force unrelated signals to be published together.

How do we associate the two streams later?

Because both carry the same type of **SampleHeader**:

timestamp
run_id
program_id
part_id
ply_id
course_id
NC block
path progress

For example:

ActualMachineState

timestamp = 12:15:10.100

run = RUN_05

ply = PLY_07

course = COURSE_12

block = 1250

TCP = ...

ActualProcessData

timestamp = 12:15:10.110

run = RUN_05

ply = PLY_07

course = COURSE_12

block = 1250

Tow17 = 42.3 N

The Spatial Analysis RXU can then associate the KPP sample with the corresponding TCP using timestamp + run/course/path context.

One refinement I would make from our earlier version: **I would not duplicate `actual_tcp` inside `ActualProcessData`** if Connectivity truly publishes TCP independently. Let the spatial-analysis component perform the timestamp/path alignment. That keeps the process message clean and avoids potentially inconsistent duplicated TCP values.

i didn't understand why two separate proto files?

You do **not need two separate `.proto` files.**

That was only an organizational suggestion, not a requirement.

The important decision we made was:

Two different protobuf messages + two different topics.

You can absolutely keep both messages in the **same `.proto` file.**

For example:

```
afp_connectivity.proto
├── ManufacturingContext
├── SampleHeader
├── Pose
├── JointState
├── TowProcessData
├── ActualMachineState
└── ActualProcessData
```

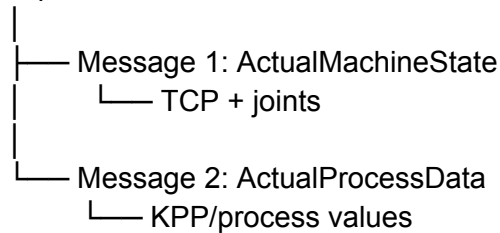
Then RXU Connectivity publishes:

```
/afp/actual_machine_state
  message type = ActualMachineState
```

```
/afp/actual_process_data
  message type = ActualProcessData
```

So:

ONE .proto file



That is probably simpler for you at this stage.

The reason I previously separated `common.proto` was only to avoid repeating common definitions later, because other applications like Spatial Analysis, Anomaly, Machine Health, etc. will also need `ManufacturingContext`, `SampleHeader`, `Pose`, and so on.

But initially, you can keep everything in one file:

```
syntax = "proto3";

package rxu.afp.connectivity;

import "google/protobuf/timestamp.proto";

message ManufacturingContext {
  string machine_id = 1;
  string product_id = 2;
  string part_id = 3;
  string run_id = 4;
  string program_id = 5;
  string ply_id = 6;
  string course_id = 7;
  optional string strip_id = 8;
  optional uint32 nc_block_id = 9;
  optional double path_progress_m = 10;
}

message SampleHeader {
  google.protobuf.Timestamp source_timestamp = 1;
  google.protobuf.Timestamp connectivity_timestamp = 2;
  uint64 sequence_id = 3;
  ManufacturingContext context = 4;
}

message Vector3 {
  double x = 1;
```

```
    double y = 2;  
    double z = 3;  
}
```

```
message Quaternion {  
    double x = 1;  
    double y = 2;  
    double z = 3;  
    double w = 4;  
}
```

```
message Pose {  
    Vector3 position = 1;  
    Quaternion orientation = 2;  
    string frame_id = 3;  
}
```

```
message JointState {  
    string joint_name = 1;  
    double position = 2;  
    optional double velocity = 3;  
    optional double effort = 4;  
}
```

```
// MESSAGE 1
```

```
message ActualMachineState {  
    SampleHeader header = 1;  
  
    Pose actual_tcp = 2;  
  
    repeated JointState joints = 3;  
  
    optional Vector3 tcp_linear_velocity = 4;  
  
    optional Vector3 tcp_angular_velocity = 5;  
  
    optional double actual_process_speed_mps = 6;  
}
```

```
// Tow-specific KPPs
```

```
message TowProcessData {  
    string tow_id = 1;
```

```

optional string channel_id = 2;

optional double tension_n = 3;

optional double temperature_c = 4;

optional double pressure_pa = 5;

optional bool tow_present = 6;

optional bool tow_broken = 7;

optional bool tow_laid = 8;
}

// MESSAGE 2
message ActualProcessData {
  SampleHeader header = 1;

  optional double compaction_force_n = 2;

  optional double compaction_pressure_pa = 3;

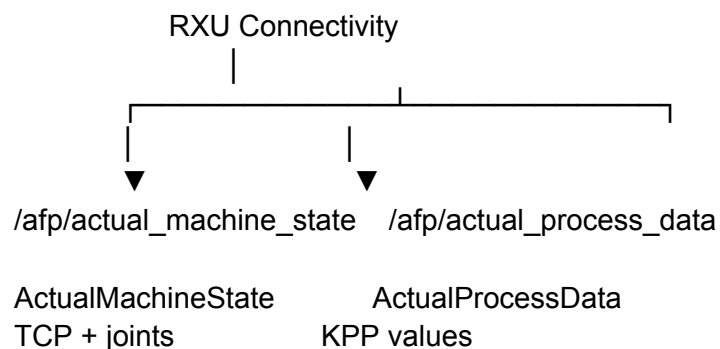
  optional double process_temperature_c = 4;

  optional double laying_speed_mps = 5;

  repeated TowProcessData tow_data = 6;
}

```

Then your architecture is simply:



So the key distinction is:

.proto file = container where message definitions are written.

protobuf message = structure of one type of data.

topic = where that message is published.

You can therefore have:

- 1 proto file
- 2 message definitions
- 2 RXU topics

For your case, I think this is the clearest arrangement. Later, if many RXU apps start reusing the common structures, we can split **ManufacturingContext**, **SampleHeader**, **Pose**, etc. into a **common.proto**. But there is no need to do that now.

can you give the proto definition or structure of the analysis componenet (RXU.App) which publishes the tcp/jpint deviation and KPP trends

will it be same topic and same proto ?

I would **not use the same topic and not use one combined message**.

You can still keep both message definitions inside the same **.proto** file, for example:

```
spatial_analysis.proto
├── SpatialDeviationResult
└── KppTrendResult
```

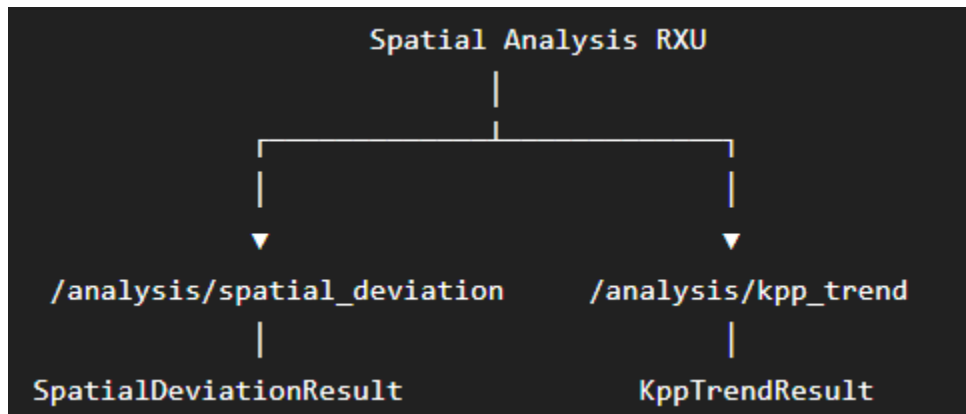
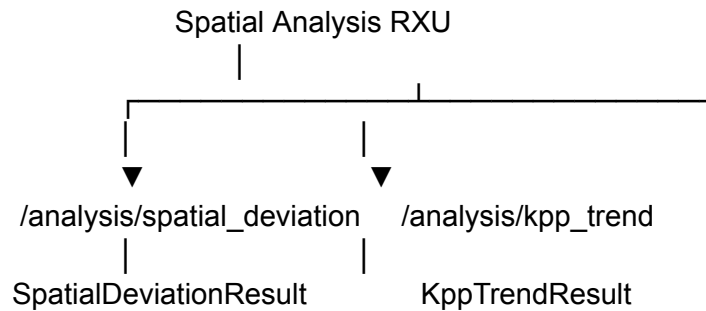
and publish them on two different topics:

```
/analysis/spatial_deviation
→ SpatialDeviationResult
```

```
/analysis/kpp_trend
→ KppTrendResult
```

That is the cleanest design because TCP/joint deviation and KPP trend are different result types.

For the RXU analysis app, conceptually:



A single `.proto` file could look like this:

```
syntax = "proto3";
```

```
package rxu.afp.analysis;
```

```
import "google/protobuf/timestamp.proto";
```

```
//
```

```
=====
```

```
====
```

```
// COMMON CONTEXT
```

```
//
```

```
=====
```

```
====
```

```
message ManufacturingContext {  
    string machine_id = 1;  
    string product_id = 2;
```

```

string part_id = 3;
string run_id = 4;
string program_id = 5;
string ply_id = 6;
string course_id = 7;

optional string strip_id = 8;
optional uint32 nc_block_id = 9;
optional double path_progress_m = 10;
}

```

```

message SampleHeader {
    google.protobuf.Timestamp source_timestamp = 1;

    google.protobuf.Timestamp analysis_timestamp = 2;

    uint64 sequence_id = 3;

    ManufacturingContext context = 4;
}

```

```

//
=====
====
// GEOMETRY
//
=====
=====

```

```

message Vector3 {
    double x = 1;
    double y = 2;
    double z = 3;
}

```

```

message Quaternion {
    double x = 1;
    double y = 2;
    double z = 3;
    double w = 4;
}

```

```

message Pose {
    Vector3 position = 1;
    Quaternion orientation = 2;

    string frame_id = 3;
}

```

```

//
=====
====
// TCP DEVIATION
//
=====
=====

```

```

message TcpDeviation {

    // Reference/programmed TCP used for comparison
    Pose programmed_tcp = 1;

    // Actual measured TCP
    Pose actual_tcp = 2;

    // actual - programmed
    Vector3 translation_error = 3;

    //  $\sqrt{dx^2 + dy^2 + dz^2}$ 
    // Unit: metres
    double position_deviation_m = 4;

    // Optional orientation error
    // Unit: radians
    optional double orientation_deviation_rad = 5;

    // Optional AFP-specific components
    optional double surface_normal_deviation_m = 6;
    optional double tangential_deviation_m = 7;
}

```

```
//
=====
====
// JOINT DEVIATION
//
=====
=====
```

```
message JointDeviation {

    string joint_name = 1;

    // rad for rotary joints, m for linear joints
    double programmed_position = 2;

    double actual_position = 3;

    // actual - programmed
    double deviation = 4;

    string unit = 5;
}
```

```
//
=====
=====
// MESSAGE 1
// SPATIAL TCP / JOINT DEVIATION RESULT
//
// Suggested topic:
// /analysis/spatial_deviation
//
=====
=====
```

```
message SpatialDeviationResult {

    SampleHeader header = 1;

    TcpDeviation tcp_deviation = 2;

    repeated JointDeviation joint_deviations = 3;
```

```

// Reference point used from programmed trajectory
optional uint64 reference_point_id = 4;

optional uint32 nc_block_id = 5;

optional double path_progress_m = 6;
}

//
=====
====
// KPP TYPES
//
=====
====

enum KppType {
    KPP_TYPE_UNSPECIFIED = 0;

    TOW_TENSION = 1;

    COMPACTION_FORCE = 2;

    COMPACTION_PRESSURE = 3;

    PROCESS_TEMPERATURE = 4;

    LAYING_SPEED = 5;
}

//
=====
====
// MESSAGE 2
// KPP TREND / VARIATION RESULT
//
// This is NOT anomaly detection.
//
// Suggested topic:
// /analysis/kpp_trend

```

```

//
=====
=====

message KppTrendResult {

    SampleHeader header = 1;

    KppType kpp_type = 2;

    // Used for tow-specific KPPs such as tension
    optional string tow_id = 3;

    string unit = 4;

    // Current KPP value
    double current_value = 5;

    // Statistics calculated over the configured trend window
    optional double moving_average = 6;

    optional double minimum = 7;

    optional double maximum = 8;

    optional double standard_deviation = 9;

    // Variation compared with selected reference/baseline
    optional double reference_value = 10;

    optional double absolute_variation = 11;

    optional double percentage_variation = 12;

    // Rate of change over trend window
    optional double trend_slope = 13;

    // Spatial position associated with this KPP sample
    optional Pose spatial_pose = 14;

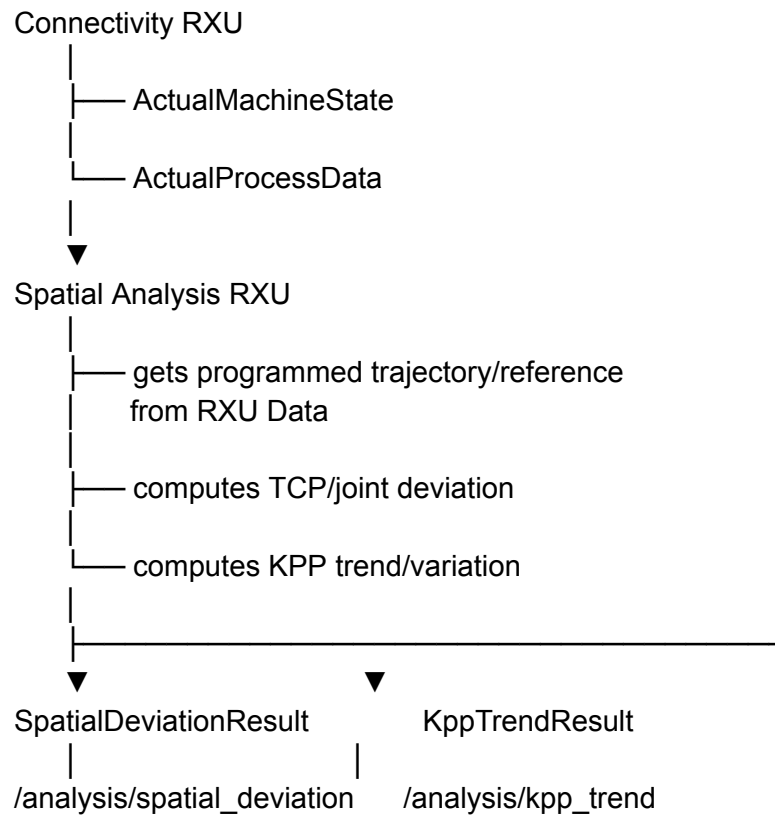
    optional double path_progress_m = 15;

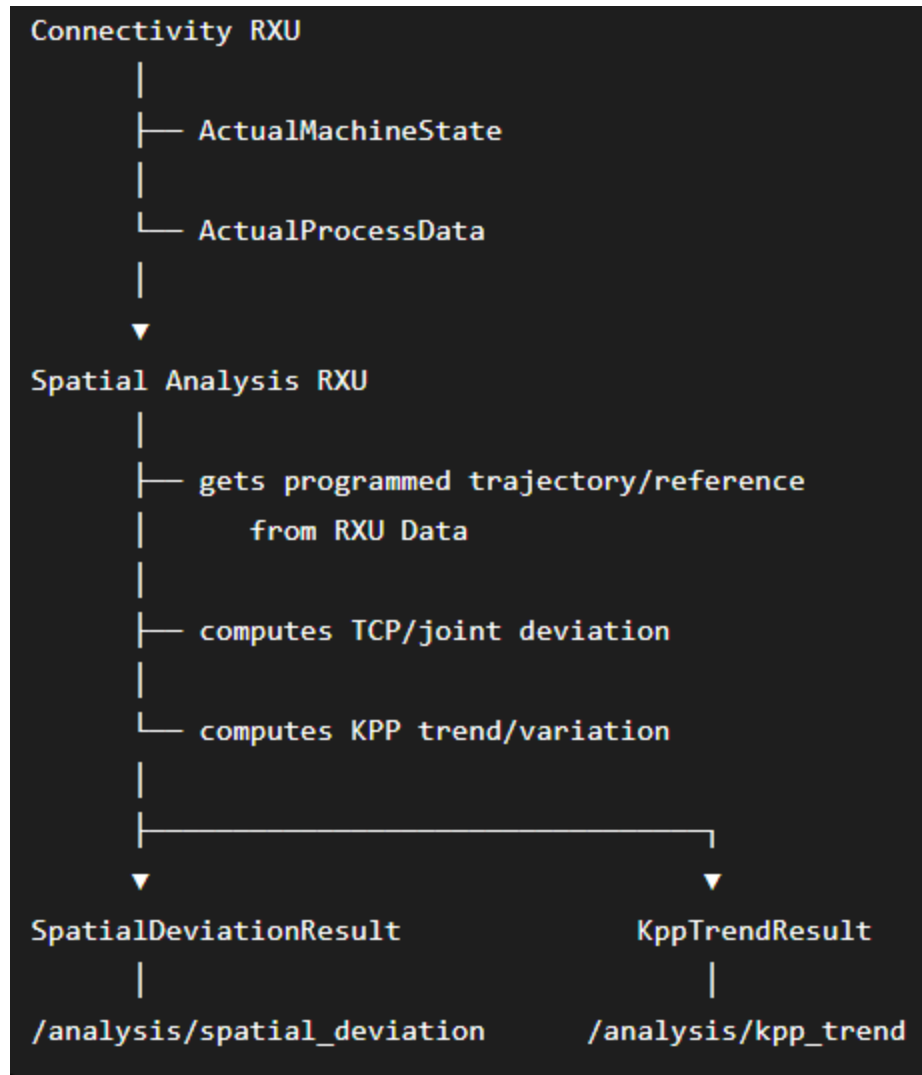
    // Trend calculation window
    google.protobuf.Timestamp window_start = 16;

```

```
google.protobuf.Timestamp window_end = 17;  
  
optional uint32 sample_count = 18;  
}
```

The analysis app then works like this:





For example, one `SpatialDeviationResult` might say:

Run = RUN_05
Ply = PLY_07
Course = COURSE_12
NC Block = 1250
Path = 2.735 m

Programmed TCP = [1.2000, 0.5000, 0.3000]
Actual TCP = [1.2012, 0.5004, 0.3003]

dx = 1.2 mm
dy = 0.4 mm
dz = 0.3 mm

Total deviation = 1.30 mm

J1 deviation = +0.002 rad

J2 deviation = -0.004 rad

...

while a separate `KppTrendResult` may say:

Run = RUN_05

Ply = PLY_07

Course = COURSE_12

Tow = TOW_17

KPP = TOW_TENSION

Current = 44.5 N

Average = 42.1 N

Min = 39.8 N

Max = 45.2 N

Variation = +4.5 N

Variation % = +11.25 %

Trend slope = +0.8 N/s

The main reason for **not combining them into one topic/message** is that they are semantically different:

`SpatialDeviationResult`

= How far actual robot execution differs from programmed execution.

`KppTrendResult`

= How a process parameter is varying over time/path.

The consumers can also be different. The 3D deviation view may care only about

`SpatialDeviationResult`, while the KPP trend plot may care only about

`KppTrendResult`.

So I would keep:

one RXU analysis app, one `.proto` file if you want, two protobuf messages, two publish topics.

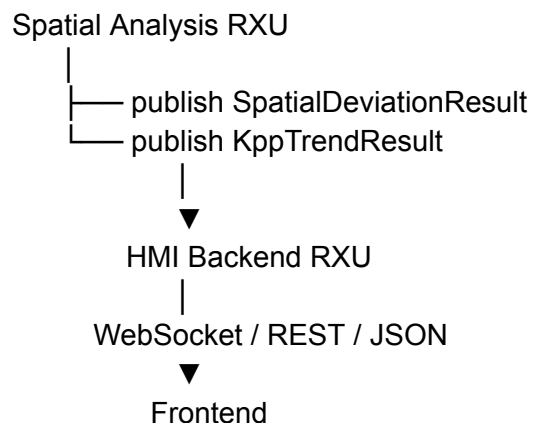
And separately, the **Process Anomaly RXU** should publish its own `ProcessAnomaly` message/topic, because KPP trend/variation is descriptive analysis, whereas anomaly detection decides whether the variation is abnormal.

if analysis component gives data like this , then how backend recieves the data and use it and pass it to front end and visualizae over 3D cad surface
also we need to write a proto file in which (service request) backend interface rxu app requests for the corresponding 3d cad surface to rxu data?
also write a proto service request raised by analysis compoenet to rxu data for the corresponding NC file
attaching the rxu data sample proto file
rxu data uses quest DB data base

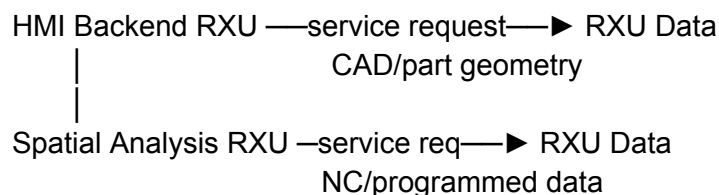
Yes. Based on the RXU Data sample proto you attached, I would keep the same overall idea:
RXU Data is accessed through service/client requests, while analysis results are published through RXU Pub/Sub.

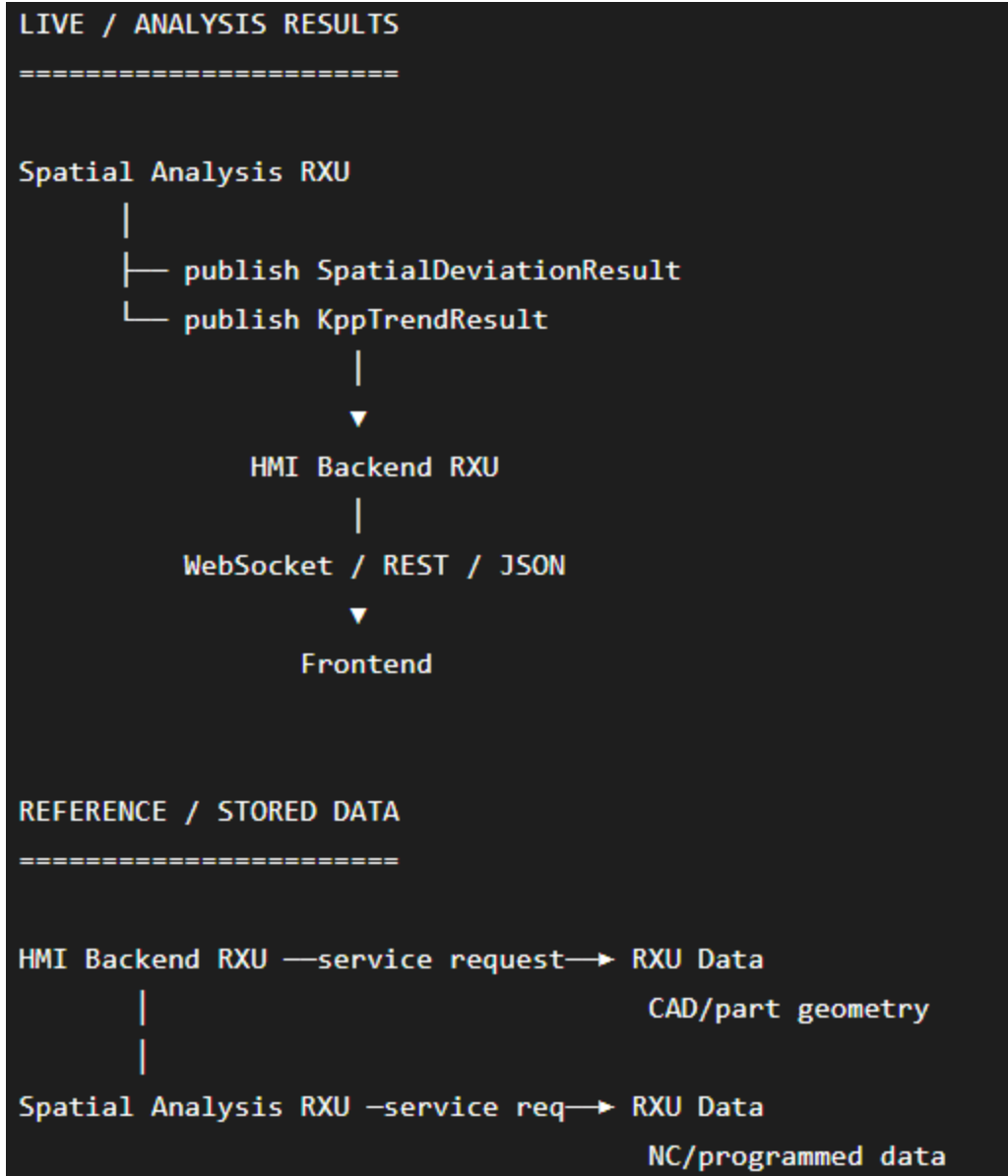
The important separation is:

LIVE / ANALYSIS RESULTS
 =====



REFERENCE / STORED DATA
 =====





So yes: **two additional RXU Data service requests are needed conceptually:**

1. HMI Backend → RXU Data: Give me the geometry/part definition for this part.
2. Spatial Analysis → RXU Data: Give me the programmed NC/trajectory for this program/course.

How the backend gets the analysis results

The backend does **not need a service request to Spatial Analysis for every live value.**

It subscribes:

HMI Backend RXU

subscribe:

/analysis/spatial_deviation



SpatialDeviationResult

subscribe:

/analysis/kpp_trend



KppTrendResult

For example:

SpatialDeviationResult

```
part_id    = PANEL_001
run_id     = RUN_05
ply_id     = PLY_07
course_id  = COURSE_12
path_progress = 2.735
actual_tcp  = ...
programmed_tcp = ...
deviation   = 1.8 mm
```

The backend receives it and forwards a frontend-friendly representation, for example JSON:

```
{
  "partId": "PANEL_001",
  "runId": "RUN_05",
  "plyId": "PLY_07",
  "courseId": "COURSE_12",
  "pathProgress": 2.735,

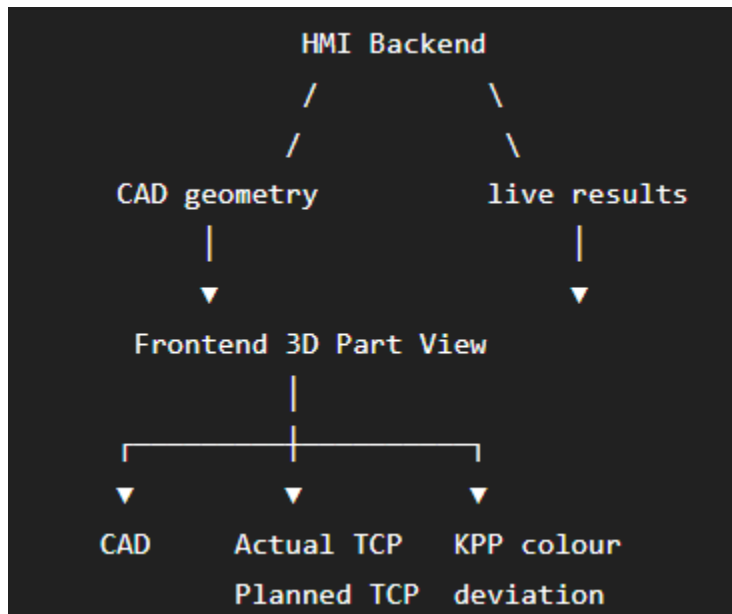
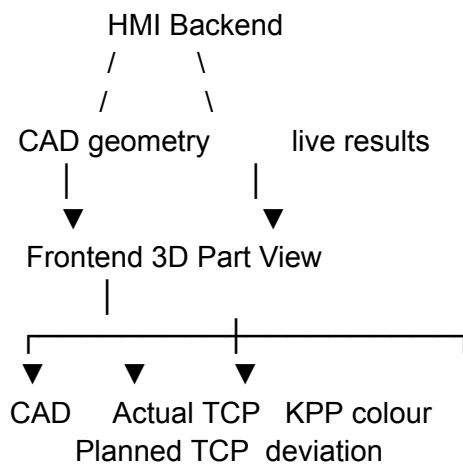
  "actualTcp": {
    "x": 1.234,
    "y": 0.455,
    "z": 0.821
  },

  "programmedTcp": {
    "x": 1.233,
    "y": 0.454,
    "z": 0.820
  }
}
```

```
},  
  
"deviationMm": 1.8  
}
```

The frontend already has the correct 3D geometry loaded and places this point/path on that surface.

So the frontend rendering becomes:



1. CAD / part geometry service

For this request, the natural lookup key is mainly:

product_id
part_id
geometry_version

Not primarily `run_id`.

Several runs may use the same panel:

```
RUN_001 ┌
RUN_002 ┤ ── PANEL_001 / REV_C
RUN_003 └
```

I would define something like this.

```
syntax = "proto3";
```

```
package rxu.data;
```

```
//
// Request sent by HMI Backend RXU to RXU Data
//
```

```
message PartGeometryRequest {
```

```
    string product_id = 1;
```

```
    string part_id = 2;
```

```
    // Optional. If absent, Data RXU can return
    // the active/latest valid geometry.
```

```
    optional string geometry_version = 3;
```

```
}
```

```
enum GeometryFormat {
```

```
    GEOMETRY_FORMAT_UNSPECIFIED = 0;
```

```
// Parametric JSON definition used by the AFP viewer
PARAMETRIC_JSON = 1;

GLB = 2;

STL = 3;

STEP = 4;

OBJ = 5;
}
```

```
message PartGeometryResponse {

    string product_id = 1;

    string part_id = 2;

    string geometry_id = 3;

    string geometry_version = 4;

    GeometryFormat format = 5;

    // Coordinate system used by the geometry.
    //
    // Important because TCP / KPP spatial coordinates
    // must eventually use the same frame.
    string frame_id = 6;

    // Preferred when geometry is stored externally
    // or RXU Data stores the reference/path.
    optional string geometry_uri = 7;

    // Can be used for a JSON parametric part definition
    // or small serialized geometry.
    optional bytes geometry_payload = 8;

    // Optional checksum/version validation.
    optional string checksum = 9;
```

```
}
```

Why **geometry_id** and **geometry_version** matter

Suppose:

Part = PANEL_001
Revision = C

The backend can cache it:

geometry_id = PANEL_001_REV_C

It doesn't need to request the CAD model every time a TCP value arrives.

Instead:

Run starts

↓

part_id = PANEL_001
geometry_version = REV_C

↓

HMI Backend requests geometry ONCE

↓

cache

↓

frontend loads geometry

Then thousands of TCP/KPP points can reference the same geometry.

2. NC/programmed trajectory request

This one is slightly different.

Although you called it “**request the corresponding NC file**”, I would avoid making Spatial Analysis dependent on an actual NC file.

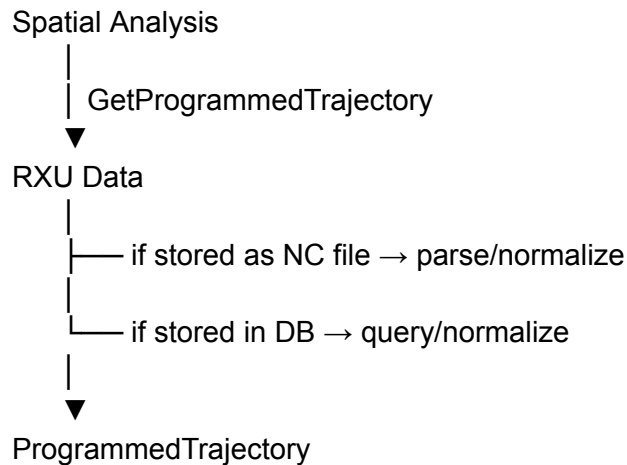
Spatial Analysis really needs:

the programmed/reference trajectory

It shouldn't care whether RXU Data obtained that trajectory from:

NC file
or
QuestDB table
or
another source

So I recommend:



That keeps the analysis algorithm independent of storage.

A proto could be:

```
message ProgrammedTrajectoryRequest {  
  
    string program_id = 1;  
  
    string part_id = 2;  
  
    // Useful if different execution runs can select  
    // different versions of the program.  
    optional string run_id = 3;  
  
    optional string ply_id = 4;  
  
    optional string course_id = 5;  
  
    // Optional requested trajectory version.  
    optional string program_version = 6;  
}
```

Then:

```
message ProgrammedJointState {  
  
    string joint_name = 1;  
  
    // rad or m depending on joint type  
    double position = 2;  
  
    string unit = 3;  
}
```

Programmed TCP:

```
message Vector3 {  
    double x = 1;  
    double y = 2;  
    double z = 3;  
}
```

```
message Quaternion {  
    double x = 1;  
    double y = 2;  
    double z = 3;  
    double w = 4;  
}
```

```
message Pose {  
    Vector3 position = 1;  
    Quaternion orientation = 2;  
    string frame_id = 3;  
}
```

And each programmed point:

```
message ProgrammedTrajectoryPoint {  
  
    uint64 sequence_id = 1;
```

```

Pose programmed_tcp = 2;

repeated ProgrammedJointState joints = 3;

optional uint32 nc_block_id = 4;

optional double path_progress_m = 5;

optional double nominal_time_s = 6;

optional double nominal_speed_mps = 7;
}

```

Complete response:

```

message ProgrammedTrajectoryResponse {

  string program_id = 1;

  string part_id = 2;

  string program_version = 3;

  optional string ply_id = 4;

  optional string course_id = 5;

  string frame_id = 6;

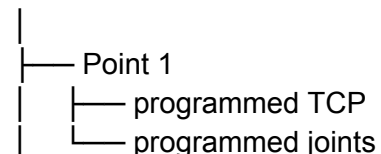
  repeated ProgrammedTrajectoryPoint points = 7;
}

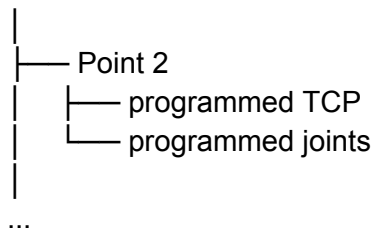
```

So the Spatial Analysis RXU gets:

ProgrammedTrajectory

Course 12





and combines that with live:

ActualMachineState

to produce:

SpatialDeviationResult

How this fits the RXU Data proto you attached

Your screenshot has a generic envelope:

```
message DataRequest {
  string topic = 1;
  string target_engine = 2;
  bytes payload = 3;
}
```

That means you don't necessarily need a separate gRPC/R XU service definition for every data type.

You can keep the generic RXU Data interface and put the **serialized typed request inside payload**.

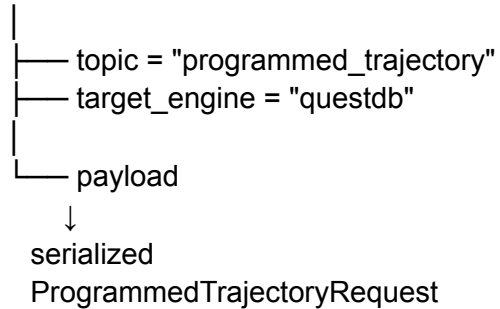
For CAD:

```
DataRequest
├── topic = "part_geometry"
├── target_engine = "questdb"
└── payload
    ↓
    serialized
```

PartGeometryRequest

For NC/programmed trajectory:

DataRequest



This is probably closest to the design visible in your existing RXU Data proto.

I would then strengthen `DataResponse` beyond just:

```
message DataResponse {
  string status = 1;
}
```

because consumers need an actual payload.

For example:

```
message DataResponse {

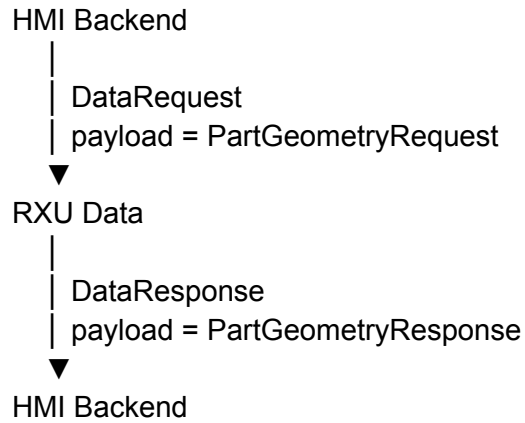
  enum Status {
    STATUS_UNSPECIFIED = 0;
    OK = 1;
    NOT_FOUND = 2;
    INVALID_REQUEST = 3;
    INTERNAL_ERROR = 4;
  }

  Status status = 1;

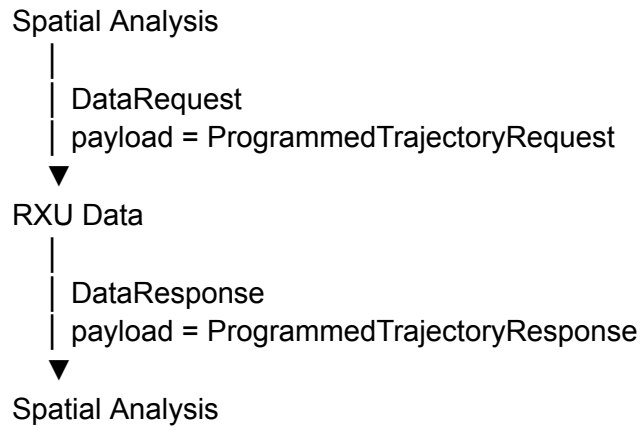
  string message = 2;

  // Serialized response protobuf.
  bytes payload = 3;
}
```

Then:

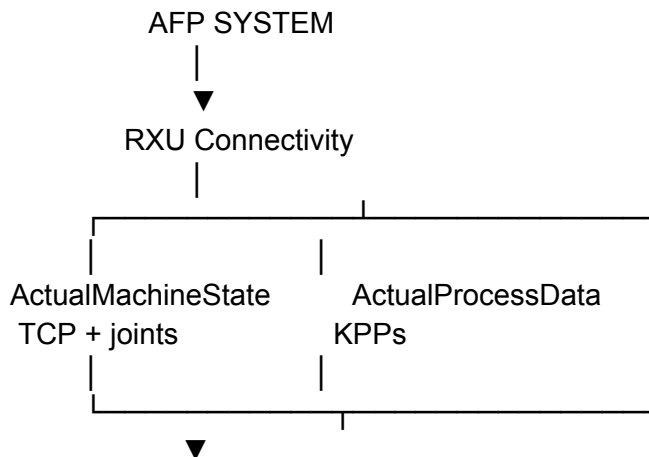


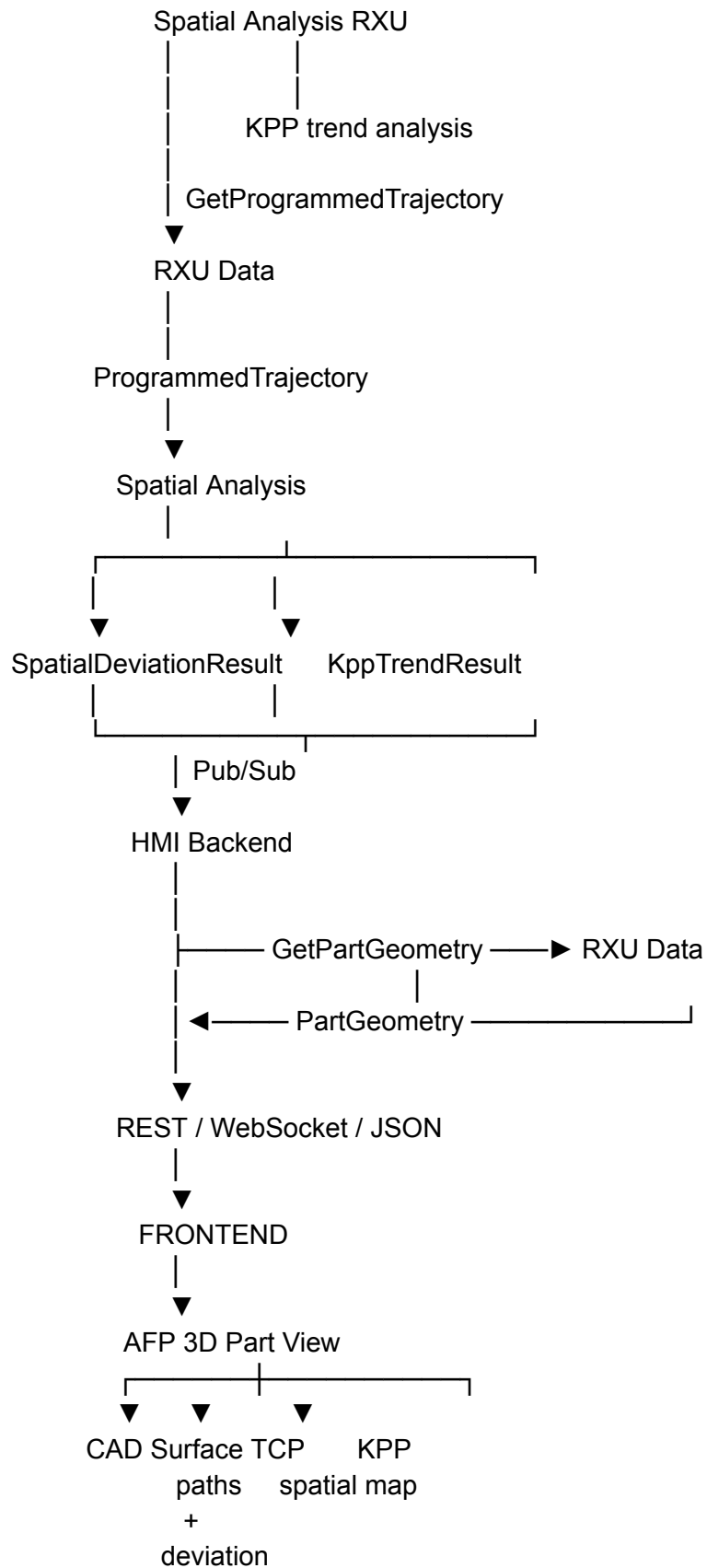
And:

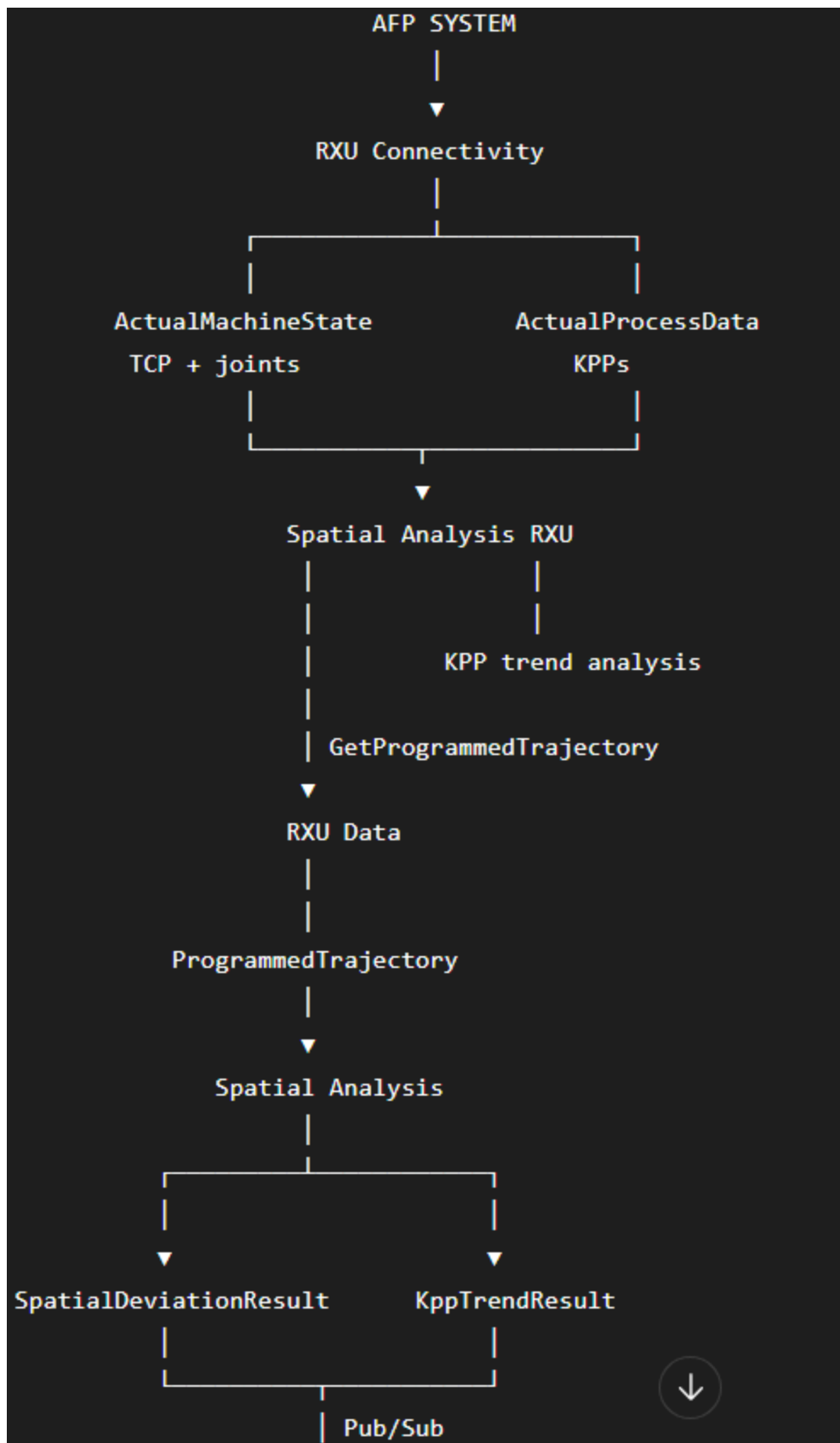


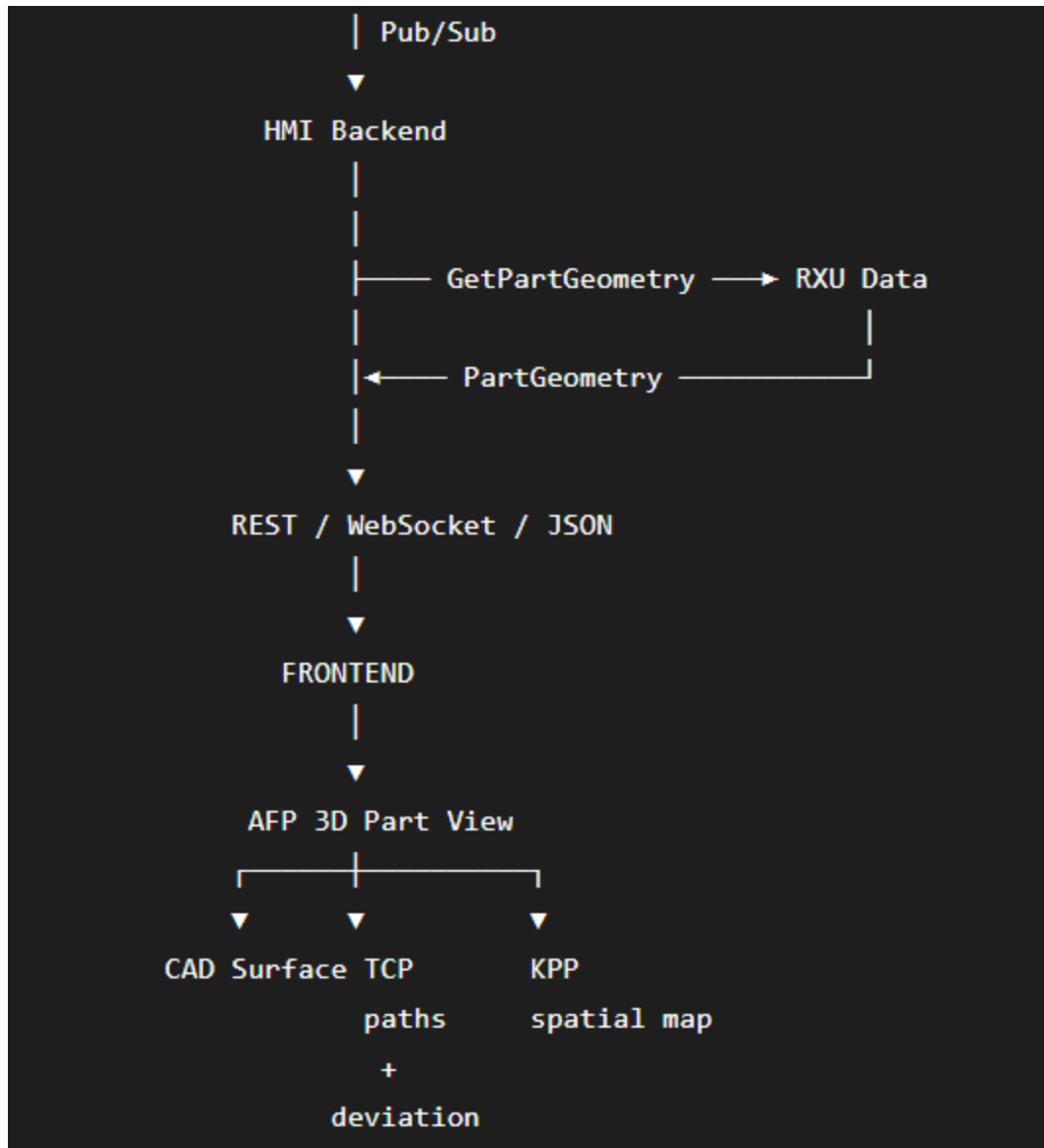
Full conceptual architecture

I would now structure your WP5.3 data flow like this:









One particularly important rule

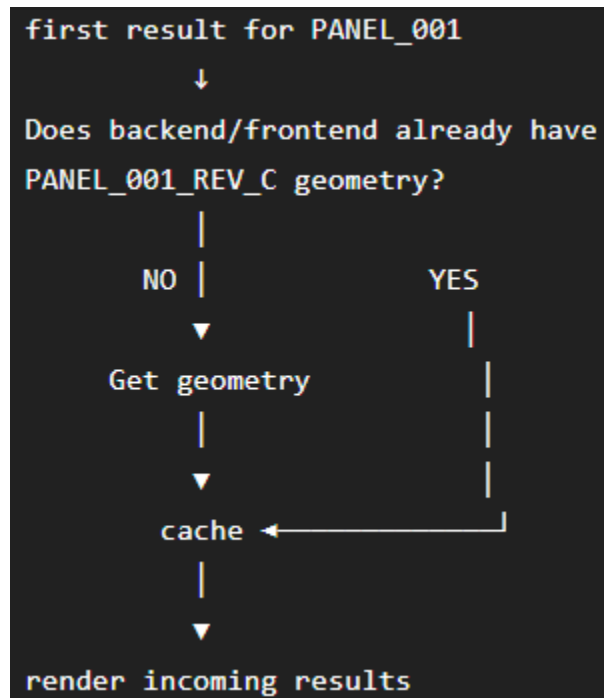
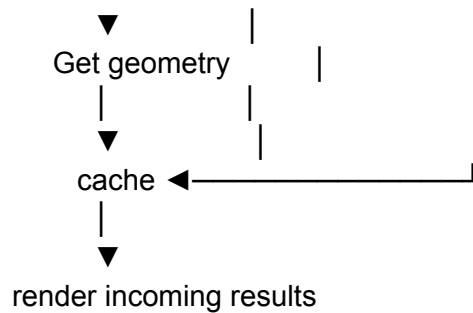
The HMI Backend should **not fetch CAD** every time it receives a deviation sample.

Instead:

first result for PANEL_001

↓
Does backend/frontend already have
PANEL_001_REV_C geometry?

NO | YES



And similarly Spatial Analysis should **load/cache the programmed trajectory once when `program_id/course_id` changes**, rather than querying QuestDB for every actual TCP sample.

That gives you a much more scalable design.

can you give the proto of the service request again?

1. RXU analysis : Spatial Analysis → RXU Data: Give me the programmed NC/trajectory for this program/course.
2. HMI Backend → RXU Data: Give me the geometry/part definition for this part.

also

any be in case of RXU data instead of separately publishing over a separate topic, they might give as a response itself

please look at the sample rxu data proto file

Yes. Looking at the RXU Data sample proto you attached, I would **not introduce separate publish topics for these two reference-data requests**.

Your sample RXU Data interface is already built around a **request → response service pattern**:

```
message DataRequest {
```

```
    string topic = 1;
```

```
    string target_engine = 2;
```

```
    bytes payload = 3;
```

```
}
```

```
message DataResponse {
```

```
    string status = 1;
```

```
}
```

```
service DataStreamService {
```

```
    rpc StreamPayloads(stream DataRequest)
```

```
        returns (stream DataResponse);
```

```
}
```

So the cleanest design is:

Spatial Analysis RXU

|

| DataRequest

| "programmed_trajectory"



RXU Data

|

| DataResponse

| contains ProgrammedTrajectoryResponse



Spatial Analysis RXU

and separately:

HMI Backend RXU

|

| DataRequest

| "part_geometry"



RXU Data

|

| DataResponse

| contains PartGeometryResponse



HMI Backend RXU

No separate Pub/Sub topic is required for either one.

The only thing missing from the current sample is that **DataResponse** currently contains only **status**. It needs a **payload** so RXU Data can return the requested content.

1. Generic RXU Data request/response

I would extend the existing definition minimally:

```
syntax = "proto3";
```

```
package rxu.data;
```

```
//  
=====
```

```
// Generic RXU Data request
```

```
//  
=====
```

```
message DataRequest {
```

```
    // Logical request/data type.
```

```
    //
```

```
    // Examples:
```

```
    // "programmed_trajectory"
```

```
    // "part_geometry"
```

```
    string topic = 1;
```

```
    // Storage/query engine used internally by RXU Data.
```

```

//
// Example:
// "questdb"
string target_engine = 2;

// Serialized request-specific protobuf message.
bytes payload = 3;

// Useful when several requests are in flight on the same stream.
string request_id = 4;
}

//
=====
====

// Generic RXU Data response

//
=====
=====

message DataResponse {

    string status = 1;

// Serialized response-specific protobuf message.

```

```

bytes payload = 2;

// Correlates this response to the original request.
string request_id = 3;

// Populated in case of error.
optional string error_message = 4;
}

//
=====
====

// Existing RXU Data service style

//
=====
====

service DataStreamService {

    rpc StreamPayloads(stream DataRequest)
        returns (stream DataResponse);
}

```

The important addition is:

bytes payload = 2;

inside `DataResponse`.

Without that, RXU Data can only say:

status = OK

but cannot actually return the trajectory or geometry.

2. Spatial Analysis → RXU Data

“Give me the programmed trajectory for this program/course”

The request-specific protobuf can be:

```
message ProgrammedTrajectoryRequest {
```

```
    // Program / NC identifier.
```

```
    string program_id = 1;
```

```
    // Part to which the program belongs.
```

```
    string part_id = 2;
```

```
    // Useful if the same program may have different execution/version context.
```

```
    optional string run_id = 3;
```

```
    optional string ply_id = 4;
```

```
optional string course_id = 5;
```

```
optional string program_version = 6;
```

```
}
```

For the response, I would return a **normalized programmed trajectory**, rather than force Spatial Analysis to understand where/how the NC is stored.

```
message Vector3 {
```

```
    double x = 1;
```

```
    double y = 2;
```

```
    double z = 3;
```

```
}
```

```
message Quaternion {
```

```
    double x = 1;
```

```
    double y = 2;
```

```
    double z = 3;
```

```
    double w = 4;
```

```
}
```

```
message Pose {  
    Vector3 position = 1;  
    Quaternion orientation = 2;  
  
    string frame_id = 3;  
}
```

```
message ProgrammedJointState {  
  
    string joint_name = 1;  
  
    // rad for rotary joint  
    // m for linear axis  
    double position = 2;  
  
    string unit = 3;  
}
```

```
message ProgrammedTrajectoryPoint {  
  
    uint64 sequence_id = 1;
```

Pose programmed_tcp = 2;

repeated ProgrammedJointState joints = 3;

optional uint32 nc_block_id = 4;

optional double path_progress_m = 5;

// If nominal timing is available.

optional double nominal_time_s = 6;

optional double nominal_speed_mps = 7;

}

message ProgrammedTrajectoryResponse {

string program_id = 1;

string part_id = 2;

optional string program_version = 3;

optional string ply_id = 4;

```

optional string course_id = 5;

// Common reference frame of the trajectory.
string frame_id = 6;

repeated ProgrammedTrajectoryPoint points = 7;
}

```

What actually goes into **DataRequest**

Spatial Analysis creates:

ProgrammedTrajectoryRequest

```

program_id = "AFP_PROGRAM_100"
part_id    = "PANEL_001"
ply_id     = "PLY_07"
course_id  = "COURSE_12"

```

serializes it and puts it into:

```

DataRequest {
  topic      = "programmed_trajectory";
  target_engine = "questdb";
  payload     = serialized ProgrammedTrajectoryRequest;
  request_id  = "REQ_001";
}

```

RXU Data receives it, queries QuestDB / resolves the underlying programmed data, and responds:

```
DataResponse {  
    status    = "OK";  
    payload   = serialized ProgrammedTrajectoryResponse;  
    request_id = "REQ_001";  
}
```

Spatial Analysis deserializes:

DataResponse.payload

↓

ProgrammedTrajectoryResponse

↓

programmed TCP

programmed joints

NC block

path progress

...

and uses that against the live [ActualMachineState](#).

3. HMI Backend → RXU Data

“Give me the part definition / 3D geometry for this part”

Request:

```
message PartGeometryRequest {  
  
    string product_id = 1;  
  
    string part_id = 2;  
  
    // Optional because HMI may ask for the currently valid version.  
    optional string geometry_version = 3;  
}
```

I would also define the geometry type explicitly:

```
enum GeometryFormat {  
  
    GEOMETRY_FORMAT_UNSPECIFIED = 0;  
  
    // Subcontractor's configurable JSON/parametric representation.  
    PARAMETRIC_JSON = 1;  
  
    GLB = 2;  
  
    STL = 3;
```

STEP = 4;

OBJ = 5;

}

Response:

message PartGeometryResponse {

string product_id = 1;

string part_id = 2;

string geometry_id = 3;

string geometry_version = 4;

GeometryFormat geometry_format = 5;

// Critical for overlaying TCP/KPP data correctly.

string frame_id = 6;

// If RXU Data returns a location/reference to the geometry.

```
optional string geometry_uri = 7;

// If RXU Data returns the actual content directly.
//
// Example:
// JSON part definition
// GLB bytes
// serialized geometry data
optional bytes geometry_payload = 8;

optional string checksum = 9;
}
```

Then HMI Backend sends:

```
DataRequest {
  topic      = "part_geometry";
  target_engine = "questdb";
  payload     = serialized PartGeometryRequest;
  request_id  = "REQ_101";
}
```

RXU Data responds:

```
DataResponse {  
    status    = "OK";  
    payload   = serialized PartGeometryResponse;  
    request_id = "REQ_101";  
}
```

So the full interaction is

A. Spatial Analysis requesting programmed data

Spatial Analysis RXU

|
| ProgrammedTrajectoryRequest
|
| wrapped inside DataRequest.payload



RXU Data

|
| query QuestDB / programmed-data source
|



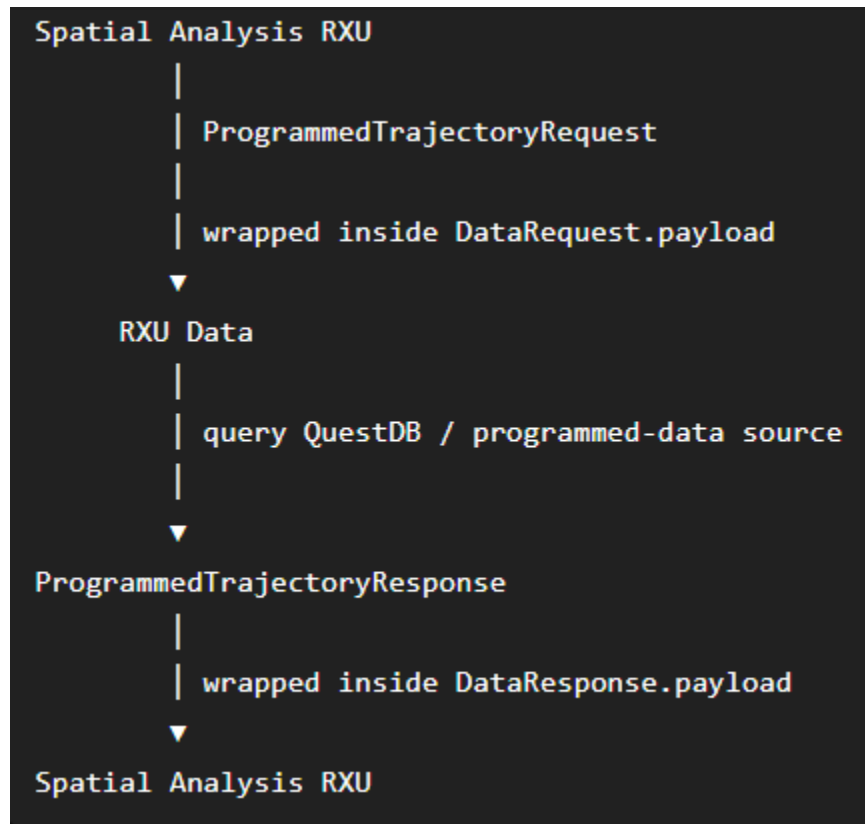
ProgrammedTrajectoryResponse

|
| wrapped inside DataResponse.payload

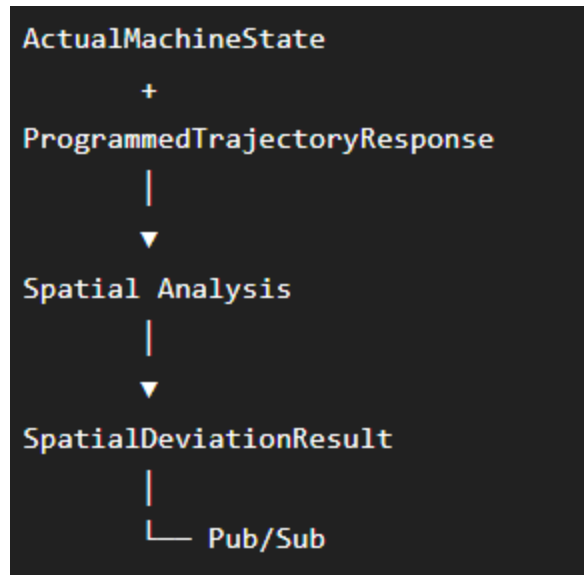


Spatial Analysis RXU

Then:

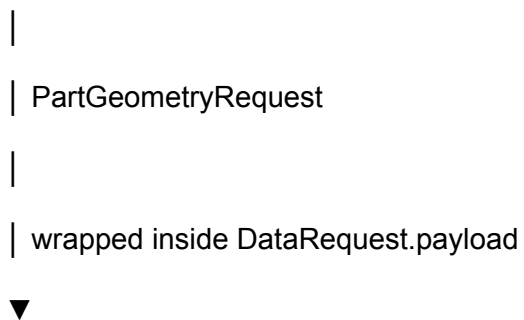


Then:



B. HMI Backend requesting geometry

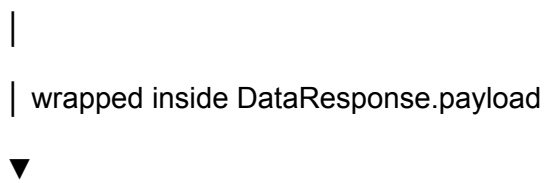
HMI Backend RXU



RXU Data



PartGeometryResponse



HMI Backend

|



Frontend

|



3D Part View

HMI Backend RXU

|

| PartGeometryRequest

|

| wrapped inside DataRequest.payload



R XU Data

|



PartGeometryResponse

|

| wrapped inside DataResponse.payload



HMI Backend

|



Frontend

|



3D Part View

This distinction is important

You now have **two different RXU communication patterns** in your architecture:

CONTINUOUS RESULTS

=====

Spatial Analysis

|

| Publish

▼

SpatialDeviationResult

Spatial Analysis

|

| Publish

▼

KppTrendResult

These are continuously generated results, so **Pub/Sub makes sense**.

But:

REFERENCE / STORED DATA

=====

Spatial Analysis

|

| Request



RXU Data



| Response



ProgrammedTrajectory

and:

HMI Backend



| Request



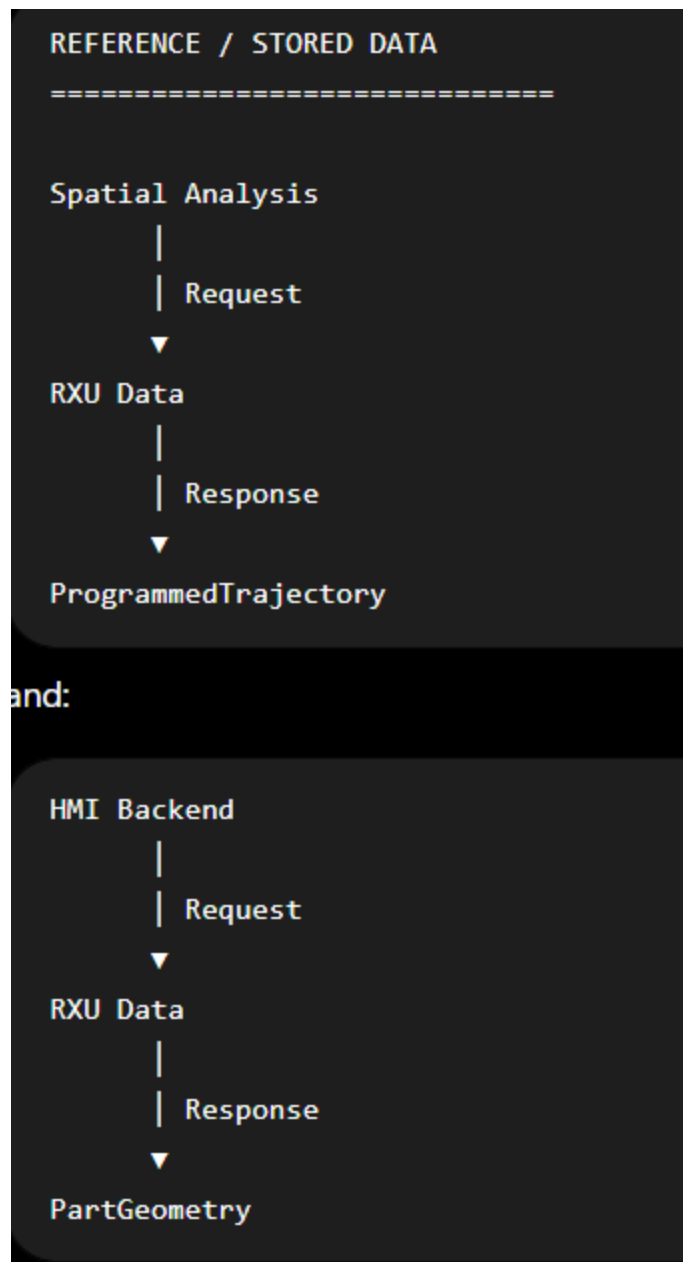
RXU Data



| Response



PartGeometry



These are **request/response**, so there's no reason for RXU Data to publish them on separate topics.

One small improvement I would recommend to the existing RXU Data proto

Because their existing service is **bidirectional streaming**, `request_id` is useful.

Without it:

Request A ————|

Request B ———|—— RXU Data

Request C ———|

Responses arrive...

Which response belongs to which request?

With:

REQ_001 → programmed trajectory

REQ_002 → geometry

REQ_003 → historical KPI

the response can simply echo:

request_id = REQ_002

That makes the generic RXU Data service much more robust.

And yes: based on the sample proto you showed, **returning the requested trajectory/geometry directly in `DataResponse.payload` is more consistent than having RXU Data respond by publishing the requested data on another topic.**

also if analysis compoennet continuously publishing to backend interface then how front use this data and do spatial super impoistion (on 3D cad surafce)?

i meant what lib front will use ?

Yes. Once the **Spatial Analysis RXU** publishes deviation/KPP results continuously and the **HMI Backend** receives them, the frontend needs two things:

1. the **3D part geometry**
2. the **spatial result values** to overlay on that geometry

The frontend library is then mainly responsible for **3D rendering and overlaying points/paths/colors**.

For your subcontractor's proposal, the most likely choice is **WebGL-based rendering**, and in practice they could implement it directly or use a library/framework on top of WebGL such as **Three.js**. Their document already mentions a WebGL renderer, so Three.js would be a very natural implementation choice.

The flow would be:

RXU Data

|

| PartGeometryResponse

▼

HMI Backend

|

| JSON/API

▼

Frontend

|

|— Load 3D panel geometry

|

|— Keep geometry in scene

▲

|

Spatial Analysis RXU

|

| SpatialDeviationResult

| KppTrendResult



HMI Backend



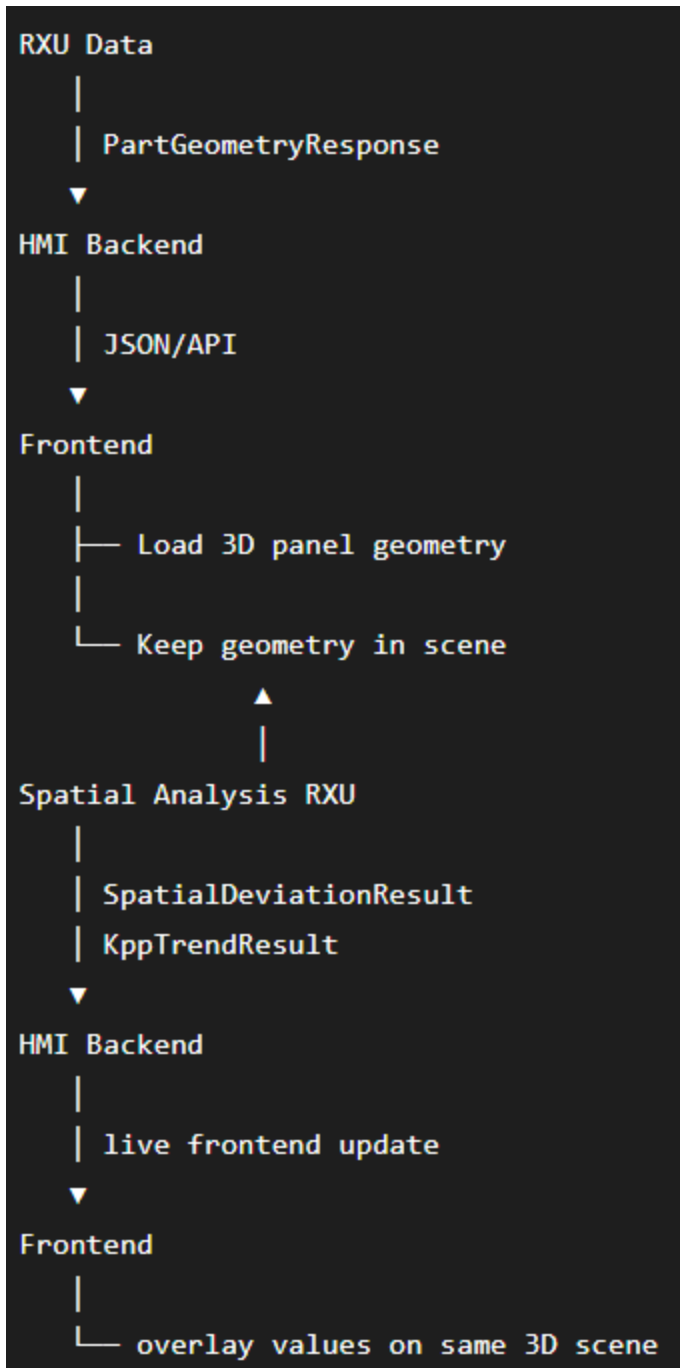
| live frontend update



Frontend



└─ overlay values on same 3D scene



For TCP deviation

The analysis result already carries spatial information, for example:

actual TCP:

x, y, z

programmed TCP:

x, y, z

deviation:

1.8 mm

frame_id:

part_frame

The frontend can draw:

Programmed trajectory → line

Actual trajectory → line

Deviation → colour / connector / marker

With Three.js, conceptually:

```
const point = new THREE.Vector3(x, y, z);
```

and add it to the same scene containing the aircraft-panel geometry.

The crucial requirement is:

CAD geometry and analysis coordinates must use the same coordinate frame.

For example:

Part geometry

frame_id = part_frame

SpatialDeviationResult

frame_id = part_frame

Then the frontend can directly superimpose them.

If the frames differ, the transformation must be supplied/applied before rendering.

For KPP spatial mapping

KPP is slightly different.

Suppose the analysis publishes:

KPP = Tow Tension

Value = 42.5 N

Ply = 7

Course = 12

Tow = 17

TCP = [x,y,z]

There are two possible frontend approaches.

Point/path based overlay

Use the TCP/spatial coordinate:

x,y,z

↓

find position on panel

↓

colour marker/path

This works well for:

- TCP deviation
- temperature values along trajectory
- pressure along trajectory
- laying speed
- anomaly markers

Tow/ribbon based overlay

This is closer to what the subcontractor proposed:

Ply 7

Course 12

Tow 17

Segment 24

Value = 42.5 N



frontend already knows geometry of

that tow ribbon/segment



colour that piece of geometry

This is especially useful for:

- tow tension
- tow present/missing
- gaps
- tow-specific quality values

Which frontend library?

For this project, I would expect one of these:

- **Three.js** — very suitable for custom industrial 3D visualization, trajectories, heatmaps, markers, WebGL geometry.
- **Babylon.js** — also capable, but less commonly chosen for engineering dashboards than Three.js.
- Direct **WebGL** — possible, but much more work and probably unnecessary.
- **vtk.js** — strong for scientific/engineering visualization, especially meshes/scalars, but may be heavier than needed.

Given the subcontractor's document explicitly says **WebGL**, I would ask them:

Which WebGL abstraction/library will be used for the AFP 3D Part View (e.g. Three.js or equivalent), and how will spatial analysis values be mapped onto the loaded geometry?

I would not prescribe Three.js unless Airbus has a technology constraint. But architecturally, **Three.js is a very sensible choice.**

The frontend does not calculate the deviation

This distinction is important:

Spatial Analysis RXU

calculates

deviation / KPP trend

↓

HMI Backend

↓

Frontend

renders it spatially

So the frontend should receive something like:

```
{  
  "partId": "PANEL_001",  
  "frameId": "part_frame",  
  "position": {  
    "x": 1.245,  
    "y": 0.426,  
    "z": 0.812  
  },  
  "tcpDeviationMm": 2.4  
}
```

and turn 2.4 mm into a visual colour/marker.

For example:

0–1 mm → normal

1–2 mm → moderate

>2 mm → high

The frontend decides **how it looks**; the backend/analysis decides **what the value actually is**.

One architectural improvement

For the frontend, don't make it search the 3D mesh geometrically for every incoming value if you can avoid it.

It is better if your backend/analysis data contains enough indexing information:

part_id

ply_id

course_id

tow_id

segment_id / path_progress

x,y,z

Then the frontend can maintain a lookup:

PLY_07 / COURSE_12 / TOW_17 / SEGMENT_24

↓

Three.js mesh/ribbon object

and update the colour directly.

That will scale much better than repeatedly doing nearest-point searches on the whole CAD mesh.

So the frontend essentially needs a **3D rendering library + a deterministic mapping between manufacturing/spatial IDs and graphical geometry**. The analytical computation itself remains outside the frontend.

also there are cases in which tcp values, joint values, deviation, KPI trends will have to internally store in a buffer by backend interface or backend interface have to query data to rxu data

eg: if the user clicked KPI trend layout and immediately switched to tcp/joint value layout, and the back to KPI layout user should be able to see the previous plot as well (say time between 4s to 10s) and vice versa in case of layout switching or tab switching

what will do in this case? backend has to keep a buffer or it has to request every time from rxu.data? which is better? because analysis component will be always publishing current values

in case if backend is getting from rxu.data (only while layout or tab switching) how backend will get the context (example IDs, time period) when it was switching?

For your case, the better design is **both**:

- **HMI Backend keeps a bounded in-memory buffer/cache for recent live data**
- **RXU Data/QuestDB remains the authoritative historical store**
- When the user switches tabs/layouts, the backend first serves from its cache; it queries RXU Data only if the requested window is older than what the cache contains or if a full historical range is requested.

I would not make every tab switch trigger a QuestDB query.

Analysis RXUs

| continuously publish



HMI Backend RXU

|

|— recent in-memory buffers

| TCP/joint

| deviation

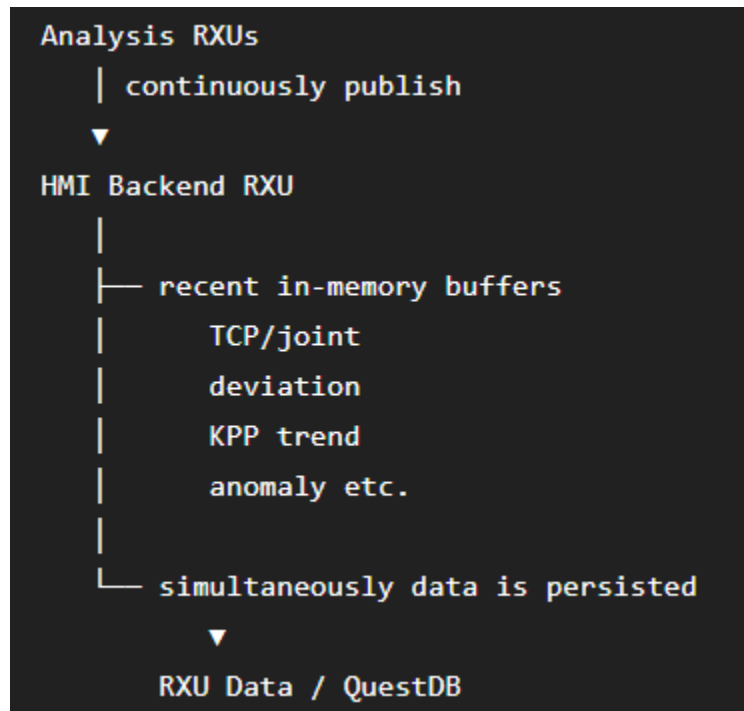
| KPI trend

| anomaly etc.

|
└─ simultaneously data is persisted



RXU Data / QuestDB



Then your example works naturally:

t = 4 s User opens KPP Trend

t = 7 s User switches to TCP/Joint

t = 10 s User returns to KPP Trend

The KPP analysis app has continued publishing during the whole period. The backend has continued receiving and buffering it even though that tab was not visible.

So when the user returns at $t = 10$ s, the backend already has:

KPP buffer:

4s

5s

6s

7s

8s

9s

10s

and the frontend can immediately redraw the whole trend rather than only showing the value arriving after 10 s.

The same applies in the other direction for TCP/joints/deviation.

I would not subscribe/unsubscribe based on the active tab

Keep the backend subscriptions alive:

Spatial Analysis —▶ HMI Backend

always subscribed

Connectivity —▶ HMI Backend

if raw TCP/joint display is needed

Anomaly RXU —▶ HMI Backend

always subscribed

```
Spatial Analysis —▶ HMI Backend
                    always subscribed

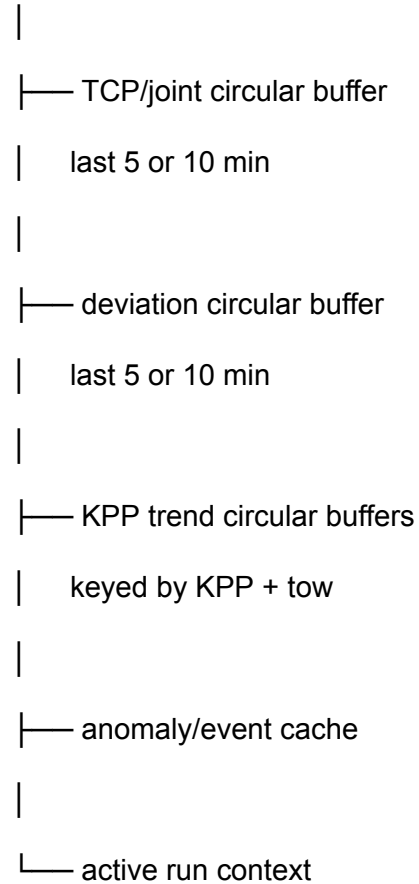
Connectivity —▶ HMI Backend
                if raw TCP/joint display is needed

Anomaly RXU —▶ HMI Backend
                always subscribed
```

Tab switching should only change **what the backend sends to that frontend session**, not whether the engineering applications are running or publishing.

A clean backend structure could be:

HMI Backend



HMI Backend

- |
- |— TCP/joint circular buffer
 - | last 5 or 10 min
- |
- |— deviation circular buffer
 - | last 5 or 10 min
- |
- |— KPP trend circular buffers
 - | keyed by KPP + tow
- |
- |— anomaly/event cache
- |
- |— active run context

For example:

KPP cache key:

run_id = RUN_05

ply_id = PLY_07

course_id = COURSE_12

kpp = TOW_TENSION

tow_id = TOW_17

Each stored sample still contains its header/context:

timestamp

product_id

part_id

run_id

program_id

ply_id

course_id

tow_id if applicable

path_progress

value

That answers your second question: **how does the backend know what context to query from RXU Data later?**

It should not infer it from the tab switch. The backend maintains an explicit **view/session filter context**.

For example, the frontend may currently have:

Selected context

part_id = PANEL_001

run_id = RUN_05

ply_id = PLY_07

course_id = COURSE_12

tow_id = ALL

time mode = CURRENT_RUN

When the user switches to KPP Trend, frontend sends something like:

```
{  
  "view": "kpp_trend",  
  "runId": "RUN_05",  
  "plyId": "PLY_07",  
  "courseId": "COURSE_12",  
  "kpp": "TOW_TENSION",  
  "timeRange": {
```

```
"start": "current-run-start",
```

```
"end": "now"
```

```
}
```

```
}
```

The HMI Backend then does:

Requested range

|



Is complete range in cache?

|



YES NO

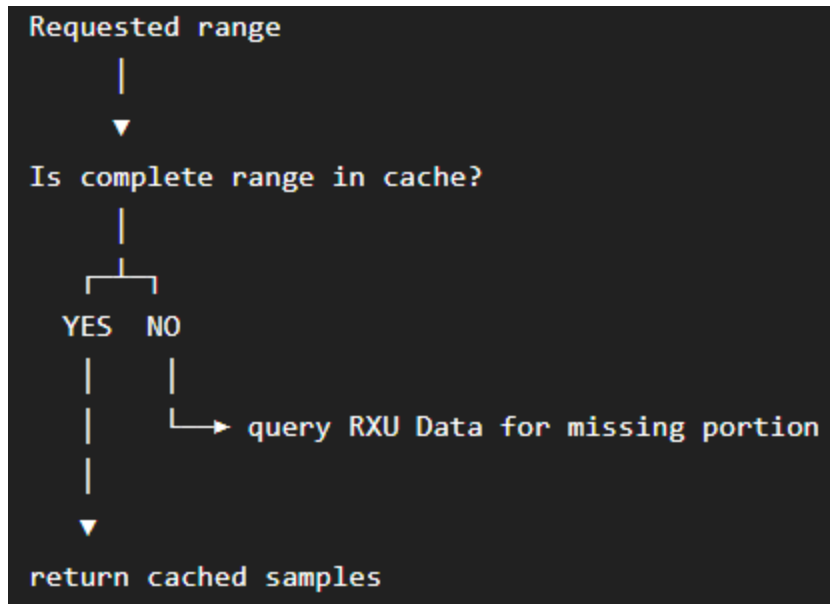
| |

| └─► query RXU Data for missing portion

|



return cached samples



For example, suppose the backend cache holds only the last **5 minutes**, but the user asks to see the whole run, which started 35 minutes ago:

QuestDB:

10:00 ————— 10:30

Backend cache:

10:25 — 10:30

Then backend can request only:

10:00 → 10:25

from RXU Data and append the recent cache:

DB history + live cache

↓

complete 10:00 → 10:30 plot

This is better than querying the entire range repeatedly.

Why not use RXU Data on every tab switch?

You could, but I would not recommend it for live screens because it introduces:

- unnecessary database queries
- additional latency when switching views
- higher QuestDB load
- possible lag between persistence and live publication
- repeated queries for data the backend already received seconds ago

For historical/review screens, querying RXU Data is absolutely appropriate.

So I would use this rule:

Situation	Source
Current value	Backend live cache
Last few seconds/minutes	Backend cache
Switch away and back	Backend cache
Current run, older portion not cached	RXU Data + cache
Previous run	RXU Data
Custom historical time range	RXU Data
Replay	RXU Data, optionally joined to live cache for active run

One subtle point: KPP trend output vs raw KPP history

If `KppTrendResult` itself contains things such as moving average, slope, min/max over a window, the backend can buffer those result messages exactly as published.

For example:

10:04:00

current = 42.1 N

moving_average = 40.8 N

10:04:01

current = 42.4 N

moving_average = 41.0 N

...

Then the frontend plots the result history.

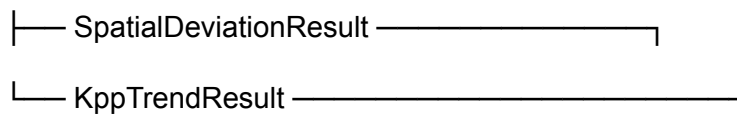
But I would also persist the underlying analysis outputs in RXU Data if you want later historical replay to reproduce the same trend.

Otherwise, after backend restart, you would lose those calculated trend-result histories and might have to recalculate them.

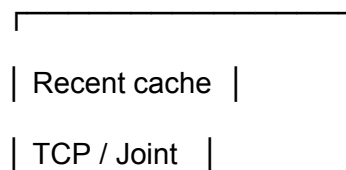
Suggested architecture

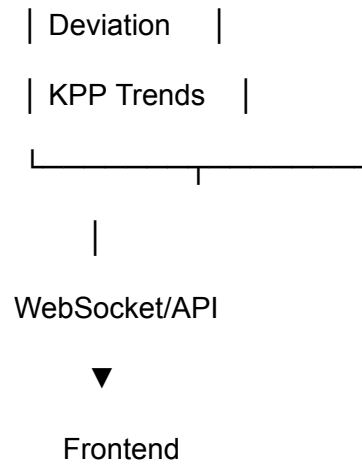
LIVE DATA PLANE

Spatial Analysis RXU



HMI Backend RXU



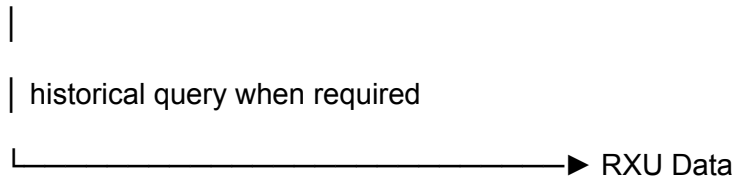


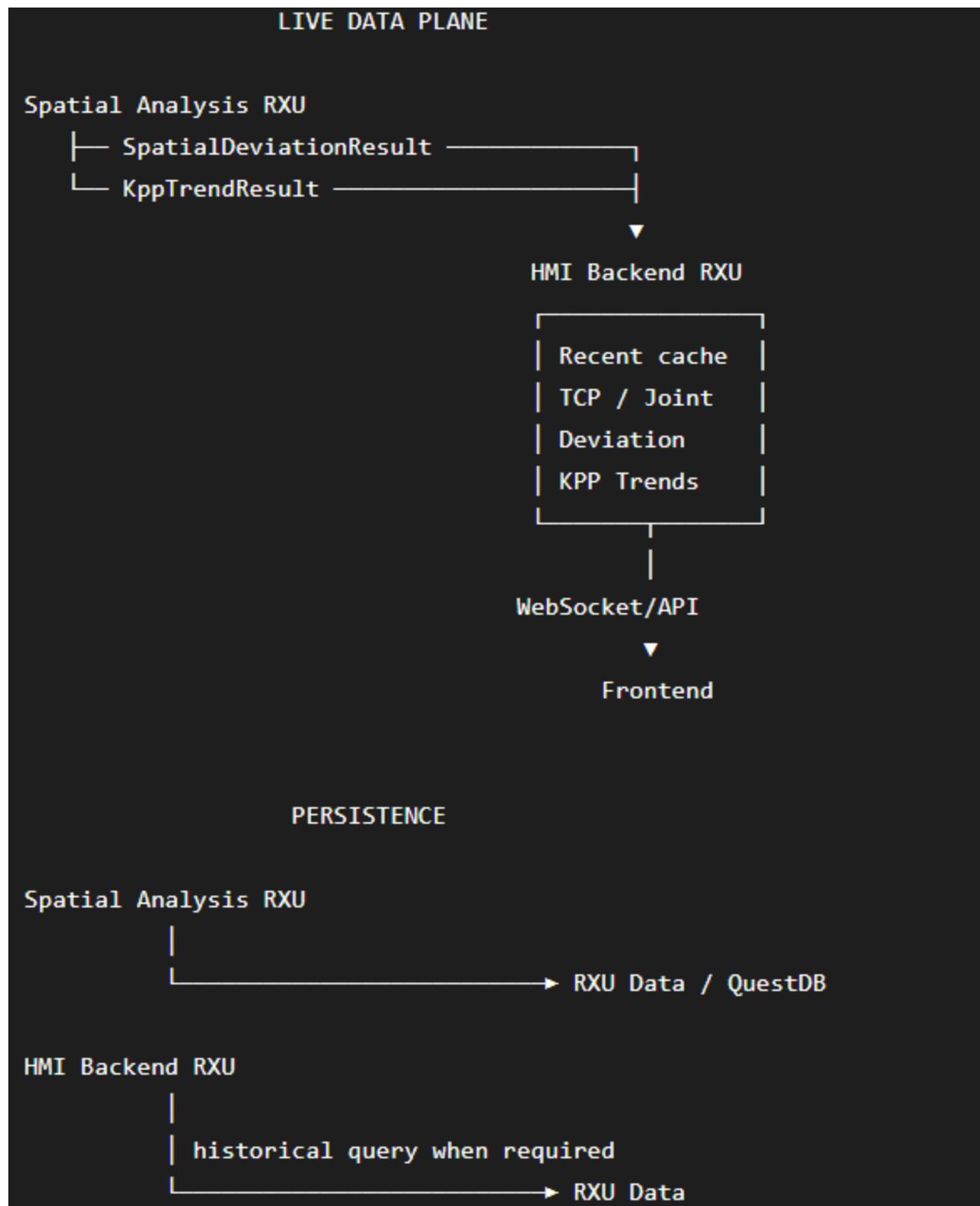
PERSISTENCE

Spatial Analysis RXU



HMI Backend RXU





If the Database RXU is itself subscribing to analysis topics, then Spatial Analysis does not need a second explicit write call; Database RXU can persist those publications.

My recommendation

Use a **hybrid cache + database design**.

I would define something like a configurable backend cache policy:

```
{  
  "live_cache": {  
    "tcp_joint_seconds": 300,  
    "deviation_seconds": 300,  
    "kpp_trend_seconds": 600,  
    "max_samples_per_stream": 50000  
  }  
}
```

That also aligns with your preference not to hard-code customer-specific behavior.

And I would explicitly make the **view context** independent of the tab:

Session / screen context

- |— product
- |— part
- |— run
- |— ply
- |— course
- |— tow
- |— time range

Tabs consume that shared context. Therefore, switching from KPP → TCP → KPP does **not lose the selected run/time/context**.

The concise architecture rule I would write in your design document is:

The HMI Backend shall continuously receive and maintain a bounded cache of recent live analysis and machine data independent of the currently active frontend view. RXU Data/QuestDB remains the authoritative persistent store. Frontend view or tab changes shall be served from the backend cache where

possible; historical or uncached ranges shall be retrieved from RXU Data using the active manufacturing and time-filter context.

That is the pattern I would choose for your WP5.3 HMI.

also can you give a complete folder structure for this?

keep in mind that rxu connectivity and rxu data is separate repository

we would like to keep

src

include

scripts

so on etc like this

i would like to keep folders like

analysis

KPP trend

time series anomaly (for process anomaly detection) --> could be separate repo in future

system identification (for behavior analysis) --> could be separate repo in future

predictive maintenance --> could be separate repo in future

i would like to CMakeLists.txt in each subfolders and src and main

i ahve given some exmple

Yes. Given your current RXU style and the fact that **RXU Connectivity** and **RXU Data** are already separate repositories, I would make a third repository dedicated to WP5.3 analysis applications, with each analysis function isolated enough that it can later be split into its own repository without major restructuring.

I would structure it like this:

RXU.Analysis/

```
|
|
|— CMakeLists.txt
|— README.md
|— .gitignore
|
|— cmake/
|   |— CompilerOptions.cmake
|   |— Dependencies.cmake
|   |— Protobuf.cmake
|
|— proto/
|   |— CMakeLists.txt
|   |— common/
|       |— AnalysisCommon.proto
|   |— spatial_analysis/
|       |— SpatialAnalysis.proto
|   |— kpp_trend/
```

```
| | └─ KppTrend.proto
| └─ time_series_anomaly/
| | └─ ProcessAnomaly.proto
| └─ system_identification/
| | └─ SystemIdentification.proto
| └─ predictive_maintenance/
|   └─ PredictiveMaintenance.proto
|
└─ analysis/
  |
  | └─ spatial_analysis/
  | | └─ CMakeLists.txt
  | | |
  | | | └─ include/
  | | |   └─ spatial_analysis/
  | | |     └─ SpatialAnalysisApp.hpp
  | | |     └─ SpatialDeviationCalculator.hpp
  | | |     └─ TrajectorySynchronizer.hpp
  | | |     └─ ProgrammedTrajectoryClient.hpp
  | | |     └─ SpatialAnalysisConfig.hpp
  | | |
  | | | └─ src/
  | | |   └─ CMakeLists.txt
  | | |   └─ main.cpp
```

```
| | | |— SpatialAnalysisApp.cpp
| | | |— SpatialDeviationCalculator.cpp
| | | |— TrajectorySynchronizer.cpp
| | | |— ProgrammedTrajectoryClient.cpp
| | |
| | |— config/
| | | |— spatial_analysis.json
| | |
| | |— scripts/
| | | |— run_spatial_analysis.sh
| | | |— validate_config.py
| | |
| | |— tests/
| | | |— CMakeLists.txt
| | | |— test_deviation.cpp
| | | |— test_synchronization.cpp
| |
| |— kpp_trend/
| | |— CMakeLists.txt
| | |— include/
| | | |— kpp_trend/
| | | | |— KppTrendApp.hpp
| | | | |— TrendCalculator.hpp
| | | | |— MovingStatistics.hpp
```

```
| | |   └─ KppTrendConfig.hpp
| | |   └─ src/
| | |   └─ CMakeLists.txt
| | |   └─ main.cpp
| | |   └─ KppTrendApp.cpp
| | |   └─ TrendCalculator.cpp
| | |   └─ MovingStatistics.cpp
| | |   └─ config/
| | |   └─ kpp_trend.json
| | |   └─ scripts/
| | |   └─ run_kpp_trend.sh
| | |   └─ tests/
| | |   └─ CMakeLists.txt
| | |   └─ test_trend.cpp
| |
| | └─ time_series_anomaly/
| |   └─ CMakeLists.txt
| |   └─ include/
| |     └─ time_series_anomaly/
| |       └─ TimeSeriesAnomalyApp.hpp
| |       └─ AnomalyDetector.hpp
| |       └─ ThresholdDetector.hpp
| |       └─ DriftDetector.hpp
| |       └─ AnomalyConfig.hpp
```

```
| | └─ src/
| |   └─ CMakeLists.txt
| |   └─ main.cpp
| |   └─ TimeSeriesAnomalyApp.cpp
| |   └─ AnomalyDetector.cpp
| |   └─ ThresholdDetector.cpp
| |   └─ DriftDetector.cpp
| | └─ config/
| |   └─ anomaly.json
| | └─ scripts/
| |   └─ run_anomaly.sh
| | └─ tests/
| |   └─ CMakeLists.txt
| |   └─ test_anomaly.cpp
| |
| └─ system_identification/
|   └─ CMakeLists.txt
|   └─ include/
|     └─ system_identification/
|       └─ SystemIdentificationApp.hpp
|       └─ BehaviourModel.hpp
|       └─ ModelEstimator.hpp
|       └─ SystemIdentificationConfig.hpp
|   └─ src/
```

```
| | | └─ CMakeLists.txt
| | | └─ main.cpp
| | | └─ SystemIdentificationApp.cpp
| | | └─ BehaviourModel.cpp
| | | └─ ModelEstimator.cpp
| | └─ config/
| | └─ system_identification.json
| | └─ scripts/
| | └─ run_system_identification.sh
| └─ tests/
| └─ CMakeLists.txt
|
| └─ predictive_maintenance/
|   └─ CMakeLists.txt
|   └─ include/
|     └─ predictive_maintenance/
|       └─ PredictiveMaintenanceApp.hpp
|       └─ HealthEstimator.hpp
|       └─ PredictiveMaintenanceConfig.hpp
|   └─ src/
|     └─ CMakeLists.txt
|     └─ main.cpp
|     └─ PredictiveMaintenanceApp.cpp
|     └─ HealthEstimator.cpp
```

```
|   |— config/
|   |   └─ predictive_maintenance.json
|   |— scripts/
|   |   └─ run_predictive_maintenance.sh
|   └─ tests/
|       └─ CMakeLists.txt
|
|— common/
|   |— CMakeLists.txt
|   |— include/
|   |   └─ rxu_analysis_common/
|   |       |— ConfigLoader.hpp
|   |       |— ManufacturingContext.hpp
|   |       |— TimestampUtils.hpp
|   |       |— RxuDataClient.hpp
|   |       └─ Logging.hpp
|   └─ src/
|       |— CMakeLists.txt
|       |— ConfigLoader.cpp
|       |— TimestampUtils.cpp
|       └─ RxuDataClient.cpp
|
|— scripts/
|   |— build.sh
```

```
| |— clean.sh
| |— format.sh
|   └— run_all.sh
|
|— config/
|   └— analysis_global.json
|
|— tests/
|   └— CMakeLists.txt
|       └— integration/
|           └— test_rxu_data_interface.cpp
|               └— test_connectivity_interface.cpp
|
└— docs/
    └— architecture.md
    └— interfaces.md
    └— README.md
```

Why I prefer this structure

The important design is that every RXU application is almost self-contained:

analysis/time_series_anomaly/

```
|— include/
|— src/
|— config/
|— scripts/
```

|— tests/

|— CMakeLists.txt

Therefore, if in six months you decide:

TimeSeriesAnomaly should become its own Git repository,

you can almost directly move that directory to:

RXU.TimeSeriesAnomaly/

|— CMakeLists.txt

|— src/

|— include/

|— config/

|— scripts/

|— tests/

|— proto/

The same applies to System Identification and Predictive Maintenance.

I would design those modules **from day one as independent RXU applications**, even if they initially share one repository.

Spatial Analysis vs KPP Trend

One refinement from our previous discussion: I would now make these **separate folders/apps**, even though earlier we discussed possibly using one Spatial Analysis RXU for both.

Why? Your repository is already evolving toward modular RXU apps:

Spatial Analysis RXU

→ TCP/joint deviation

→ trajectory synchronization

→ spatial positioning

KPP Trend RXU

→ moving average

→ min/max

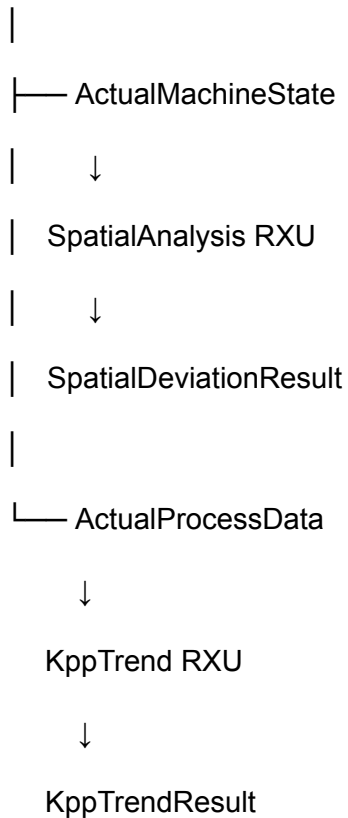
→ variation

→ trend slope

These have different responsibilities and can evolve independently.

So:

Connectivity



And:

ActualProcessData



TimeSeriesAnomaly RXU



ProcessAnomaly

This is cleaner than one application becoming:

SpatialAnalysis

- + deviation

- + trend

- + anomaly

- + behaviour modelling

- + ...

which will eventually become too large.

Top-level CMakeLists.txt

Following the style visible in your RXU.Control example, I would keep the top-level CMake very small:

```
cmake_minimum_required(VERSION 3.10)
```

```
project(RXU.Analysis LANGUAGES CXX)
```

```
set(CMAKE_CXX_STANDARD 17)
```

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
find_package(rxu_core REQUIRED)
```

```
add_subdirectory(common)
```

```
add_subdirectory(proto)
```

```
add_subdirectory(analysis/spatial_analysis)
```

```
add_subdirectory(analysis/kpp_trend)
```

```
add_subdirectory(analysis/time_series_anomaly)
```

```
add_subdirectory(analysis/system_identification)
```

```
add_subdirectory(analysis/predictive_maintenance)
```

```
option(BUILD_TESTS "Build RXU Analysis tests" ON)
```

```
if(BUILD_TESTS)
```

```
    enable_testing()
```

```
    add_subdirectory(tests)
```

```
endif()
```

Spatial Analysis CMakeLists

At:

```
analysis/spatial_analysis/CMakeLists.txt
```

I would only delegate to its source/tests:

```
add_subdirectory(src)
```

```
option(BUILD_SPATIAL_ANALYSIS_TESTS
```

"Build spatial analysis tests"

ON)

if(BUILD_SPATIAL_ANALYSIS_TESTS)

add_subdirectory(tests)

endif()

Then:

analysis/spatial_analysis/src/CMakeLists.txt

contains the real executable:

set(SPATIAL_ANALYSIS_SOURCES

main.cpp

SpatialAnalysisApp.cpp

SpatialDeviationCalculator.cpp

TrajectorySynchronizer.cpp

ProgrammedTrajectoryClient.cpp

)

add_executable(rxu_spatial_analysis

\${SPATIAL_ANALYSIS_SOURCES}

)

target_include_directories(

rxu_spatial_analysis

PRIVATE

```
    ${CMAKE_CURRENT_SOURCE_DIR}/../include
)
```

```
target_link_libraries(
    rxu_spatial_analysis
    PUBLIC
    ${rxu_core_LIBS}
    rxu_core_common
    rxu_core
    rxu_proto
    rxu_analysis_common
    rxu_analysis_proto
    spdlog::spdlog
)
```

This matches the style you showed in your existing `RXU.Control` CMake.

KPP Trend CMake

`analysis/kpp_trend/CMakeLists.txt`

```
add_subdirectory(src)

if(BUILD_TESTS)
    add_subdirectory(tests)
endif()
```

and:

analysis/kpp_trend/src/CMakeLists.txt

```
set(KPP_TREND_SOURCES
```

```
    main.cpp
```

```
    KppTrendApp.cpp
```

```
    TrendCalculator.cpp
```

```
    MovingStatistics.cpp
```

```
)
```

```
add_executable(rxu_kpp_trend
```

```
    ${KPP_TREND_SOURCES}
```

```
)
```

```
target_include_directories(
```

```
    rxu_kpp_trend
```

```
    PRIVATE
```

```
        ${CMAKE_CURRENT_SOURCE_DIR}/../include
```

```
)
```

```
target_link_libraries(
```

```
    rxu_kpp_trend
```

```
    PUBLIC
```

```
        ${rxu_core_LIBS}
```

```
        rxu_core_common
```

```
rxu_core
rxu_proto
rxu_analysis_common
rxu_analysis_proto
spdlog::spdlog
)
```

Why I added **common/**

There will be code that you don't want to duplicate across five applications.

For example:

For example:

RxuDataClient

↓

used by Spatial Analysis

used by System Identification

used by Predictive Maintenance

Similarly:

JSON ConfigLoader

Timestamp conversion

Manufacturing context handling

logging helpers

So these belong here:

common/

|— include/

|— src/

and produce:

rxu_analysis_common

as a library.

For example:

add_library(

 rxu_analysis_common

 src/ConfigLoader.cpp

 src/TimestampUtils.cpp

 src/RxuDataClient.cpp

)

target_include_directories(

 rxu_analysis_common

 PUBLIC

 \${CMAKE_CURRENT_SOURCE_DIR}/include

)

target_link_libraries(

 rxu_analysis_common

 PUBLIC

 \${rxu_core_LIBS}

 rxu_core

```
rxu_proto
spdlog::spdlog
)
```

Why a separate **proto/** directory?

I would keep all analysis **output-interface definitions** together:

proto/

- |— common/
- |— spatial_analysis/
- |— kpp_trend/
- |— time_series_anomaly/
- |— system_identification/
- |— predictive_maintenance/

For example:

SpatialAnalysis.proto

→ SpatialDeviationResult

KppTrend.proto

→ KppTrendResult

ProcessAnomaly.proto

→ ProcessAnomaly

SystemIdentification.proto

→ BehaviourModelResult

PredictiveMaintenance.proto

→ PredictiveMaintenanceResult

But note an important architectural rule:

Do not copy **RXU Connectivity and **RXU Data** proto files into this repository and start maintaining separate versions.**

Those interfaces should come from their owning repositories/package dependencies:

RXU.Connectivity repo



connectivity protobuf package



RXU.Analysis uses it

RXU.Data repo



data service protobuf package



RXU.Analysis uses it

Otherwise you will eventually have:

RXU.Data/DataRequest.proto version A

RXU.Analysis/copied/DataRequest.proto version B

and integration becomes painful.

Configuration folder is important for your project

Based on your discussions about Segula and your desire to avoid hard-coding, I'd make configuration a first-class concept.

For Spatial Analysis:

```
{  
  "application": {  
    "name": "rxu_spatial_analysis"  
  },  
  
  "subscriptions": {  
    "machine_state": "/afp/actual_machine_state"  
  },  
  
  "publishers": {  
    "spatial_deviation": "/analysis/spatial_deviation"  
  },  
  
  "rxu_data": {  
    "programmed_trajectory_service": "programmed_trajectory"  
  },  
}
```

```
"synchronization": {  
  "use_nc_block": true,  
  "use_path_progress": true,  
  "fallback_nearest_point": true  
},
```

```
"units": {  
  "position": "m",  
  "angle": "rad"  
}  
}
```

Then you are not putting things like:

```
const std::string TOPIC =  
  "/analysis/spatial_deviation";
```

throughout source code.

Similarly, KPP Trend:

```
{  
  "subscriptions": {  
    "process_data": "/afp/actual_process_data"  
  },
```

```
"publishers": {  
  "kpp_trend": "/analysis/kpp_trend"  
},
```

```
"trend": {  
  "window_seconds": 10,  
  "minimum_samples": 5  
},  
  
"kpps": {  
  "tow_tension": {  
    "enabled": true  
  },  
  "compaction_force": {  
    "enabled": true  
  },  
  "temperature": {  
    "enabled": true  
  }  
}  
}
```

That fits the architecture principle you have been emphasizing:

generic behavior in code; project/customer-specific configuration outside the code.

I would keep **main.cpp** very small

For every app:

```
#include "spatial_analysis/SpatialAnalysisApp.hpp"
```

```
int main(int argc, char** argv)
```

```
{
```

```
    SpatialAnalysisApp app(argc, argv);
```

```
    return app.run();
```

```
}
```

`main.cpp` should not contain analysis logic.

Instead:

`main.cpp`

↓

`SpatialAnalysisApp`

|— initialize RXU Core

|— create subscriber

|— create publisher

|— create RXU Data client

|— load config

|— run/spin

And actual mathematics belongs in:

`SpatialDeviationCalculator.cpp`

`TrajectorySynchronizer.cpp`

This separation will make unit testing much easier.

Final structure I would choose

At the highest level:

RXU.Analysis

```
|
|
|— analysis
|
| |— spatial_analysis
|
| |— kpp_trend
|
| |— time_series_anomaly
|
| |— system_identification
|
| |— predictive_maintenance
|
|
|— common
|
|— proto
|
|— config
|
|— scripts
|
|— tests
|
|— docs
|
|
|— CMakeLists.txt
|
|— README.md
```

This gives you a **single repository today**, but each analysis application already has a repository-like structure. So if `time_series_anomaly`, `system_identification`, or `predictive_maintenance` becomes large enough later, you can extract it into its own Git repo with very little redesign.

